
# AWS Academy Cloud Foundations – Configuring Identity and Access Management, Virtual Private Cloud, and Elastic Compute Cloud

Yuuki Burleigh, CCNA

# Lab 1 – Introduction to AWS IAM

**Purpose**

This lab helps to develop an understanding of how to secure the cloud using Identity based access policies. Basic access security concepts covered in the lab contribute to developing an understanding of how to secure access in networks beyond the cloud and into traditional networks and inside applications. The cloud security concepts covered in the lab also transfer to how to secure other resources within the cloud, like the best practice of least privilege a practice used to limit access to the minimum required resources for the action or role.

**Background**

Amazon IAM or Identity Access Management is a way to secure access to resources within AWS. IAM uses username and password login credentials to prove a client's identity and give the specified access that IAM specifies for the provided login credentials. For example, if someone were hired to optimize S3 lifecycle policies within an organization, best practice for the organization would be to provide login credentials that IAM specifies can only alter S3 lifecycle policies and nothing else. This practice of creating IAM users that only have permission to access or edit the specific resources they need within the cloud is called the principle of least privilege.

The principle of least privilege is an important concept beyond just IAM policies and best practices in the cloud. It is a basic access security concept to apply to traditional networks, applications, and even in everyday life. Providing the minimum amount of access required allow for companies and organizations that use third party services and segmentation of operations prevents extensive damage within the organization when an employee or hired third party tries to abuse their provided credentials, as the perpetrator is only given the access to resources they require. An IBM report states that 83% of organizations reported in 2024 that they had an insider attack, this further emphasizes the importance of practicing the principal of least privilege as the people who attack organizations and companies from the inside already have access to resources, and being able to limit their access to only what they were originally instead to do will prevent their ability to bring down the company or organization.

Using IAM, one AWS account can have multiple users under it, each with access to different resources. The level of specificity that IAM can achieve is very granular and fine-tuned, allowing organizations and companies to follow the principle of least privilege very closely. IAM uses IAM policies to achieve this effect. These policies apply to virtually all AWS services and resources; you can limit access to simply reading configurations or editing all services. These policies are written in a JSON (JavaScript Object Notation) format, AWS has JSON code for every resource within AWS which allows for the elevated level of granularity that allows for organizations to follow to principle of least privilege very closely. The JSON statements are formatted as permission statements, allowing access rather than denying things. If something that is not explicitly allowed is attempted on a user's account, it will be implicitly denied.

Another related service within AWS is AWS Organizations. AWS Organizations allow for large Organizations to create groups of users and limit access to resources using SCPs or Service Control Policies to explicitly deny that explicitly allow.

# Lab 2 – Build Your VPC and Launch a Web Server

**Purpose**

This lab helps to develop an understanding of how to set up and connect resources within AWS using a Virtual Private Network or VPC through launching a web server in an EC2 Instance. This lab also contributes to developing an understanding of how logically isolated networks and containerized applications function and communicate with other resources in real time.

**Background**

A VPC or Virtual Private Cloud is a logically isolated network hosted in AWS that sits inside a singular AWS region but can be spread across multiple availability zones or AZs. Each VPC comes with 2 subnets, a public subnet, and a private subnet. Each subnet is allocated to an IP address range which any resources within it will be assigned one of the IP addresses within the range. The public subnet has a public IP address range so resources outside the VPC can be accessed by resources inside it, access from the outside is also allowed. Resources from the private subnet can be accessed if within the same VPC but cannot be accessed from outside the VPC without being configured, if a NAT gateway and route to the gateway is configured for the private subnet in the VPC the resources inside the subnet can initiate outbound connection but cannot be accessed from the outside. In most use cases, the private subnet is used for processing requests from the public subnet who gets requests from users.

Along with subnets, when creating a VPC it automatically comes with an Internet Gateway, Route Table, Security Group, and a Network ACL. The internet gateway is automatically installed within the VPC to provide internet access to the resources within the public subnet and does not need a NAT gateway like the private subnet as all resources in the public subnet are assigned a public IP address. Along with the internet gateway, the main route table is installed in the VPC, which comes with a default route that points all traffic to the Internet Gateway if the destination is not within the VPC. The Route table comes with a default Local Route that handles all traffic within the VPC subnets. A default security group that applies to the VPC is also created which allows for access control on what can interact with the VPC. The security group can be used in tangent with IAM policies to allow or deny access to editing the VPC using IAM. A network ACL is like a security policy or IAM policy but only applies to the VPC network as ACls use IP addresses for Access Control.

# Lab 3 – Introduction to Amazon EC2

**Purpose**

This lab builds the understanding of what EC2 instances are and how they are fundamental to providing any usable service or solution in the AWS cloud. EC2 instances are the basic units of computing in the cloud. This lab helps develop that understanding and serves as an introductory step to understanding autoscaling and elastic load balancing.

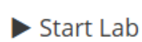
**Background**

An EC2 instance is a basic cloud computing unit to process information for cloud solutions. EC2 instances can vary in size and type, with each type/size varying in cost. Depending on the use case, the typer of EC2 instance to choose for the cloud solution will vary. T/M EC2 type is good for general purpose tasks, Type C are compute optimized and should be used for high-performance processes which, or solutions that need to deliver results fast. Type I/D are storage optimized and should be used for solutions that require fast performance and large datasets.

EC2 instances have lots of flexibility in purchasing options, On-Demand Instances, Spot Instances, and Reserved Instances. On-demand instances are the most basic and are paid by the second and require no minimum lease or any level of commitment. Spot Instances tend to be the most cost effective as they utilize unused EC2 capacity that other people are not using, although only 5% of spot instances are terminated monthly, these instances can be cut off by the providers only with a 2-minute warning or be cut off if you were outbid for the spot instance. With this drawback spot instances can save up to 90% in costs. Reserved instances are reliable but require long term commitment for typically 1-3 years but typically can save up to 72% in costs. AWS also allows for Dedicated Instances and Dedicated Hosts as well. A Dedicated Instance runs hardware dedicated to a single customer and provides isolation if needed for compliance and regulatory needs. A Dedicated Host takes that step further and dedicates a full physical server to a single customer, offering complete control over where you place your instance in the server and is beneficial for compliance and scenarios where you need your own license.

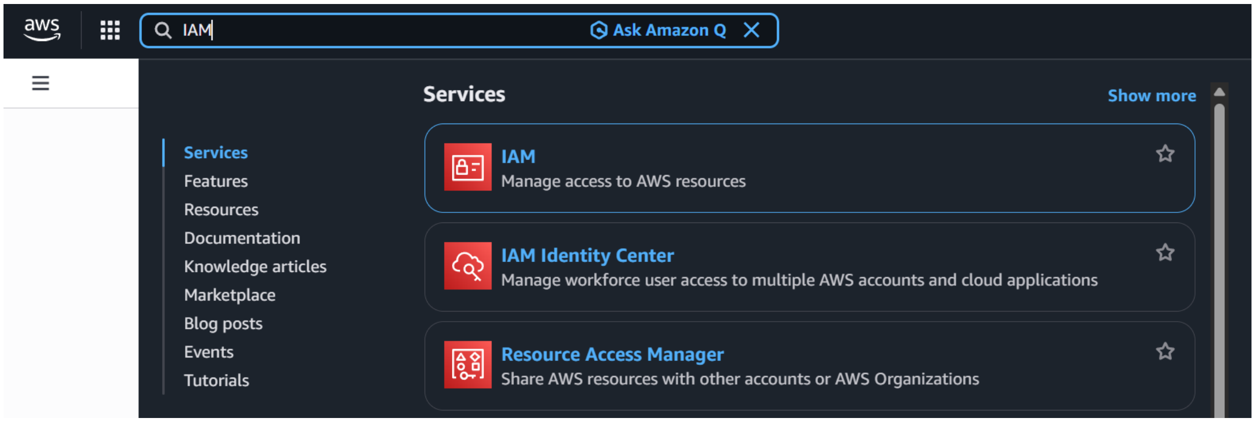
# Lab 1 – Introduction to AWS IAM - Walkthrough

Navigate to Modules in AWS Academy and finish all the instructional content leading up to the first lab

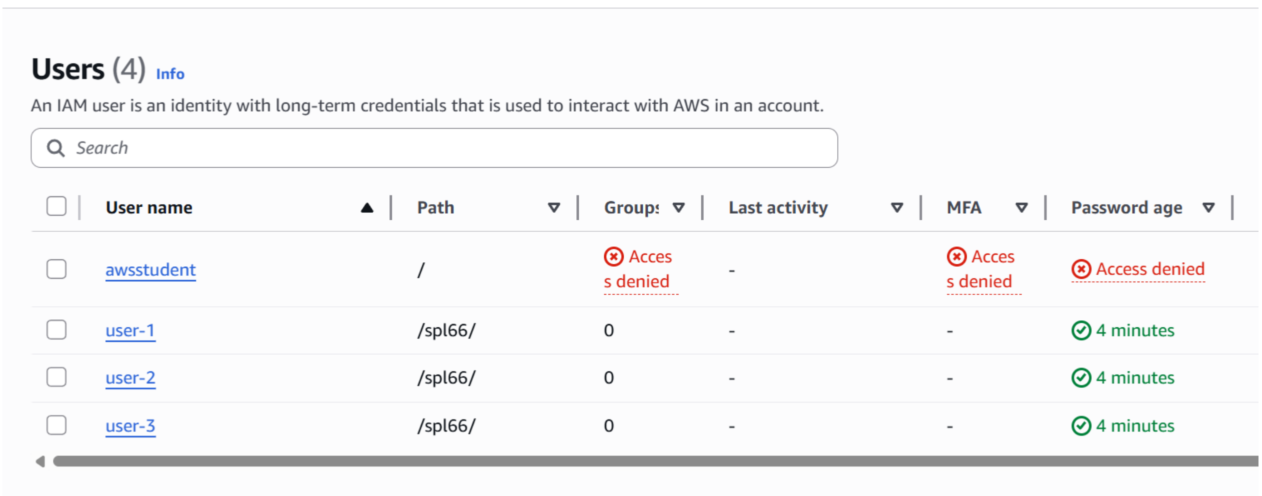
Then, click on the lab and press start lab: 


Then, click “AWS” once the icon turns green: 

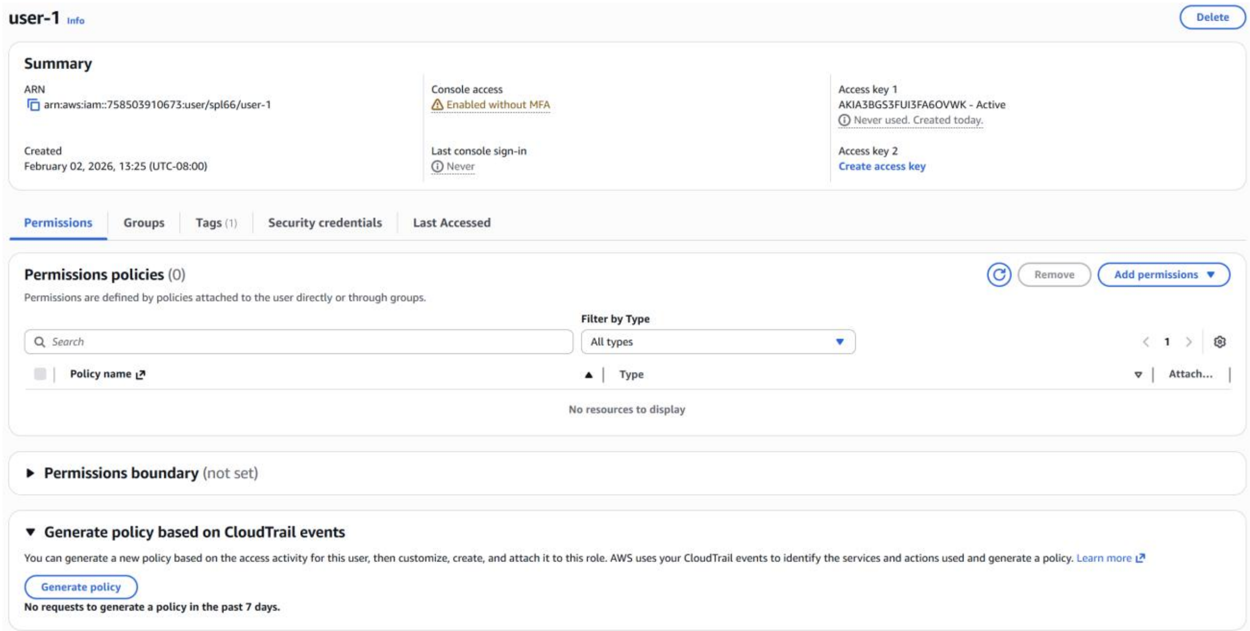

You are now in the AWS console. Then, click the search box in the top left of the AWS console and search IAM and click on it.



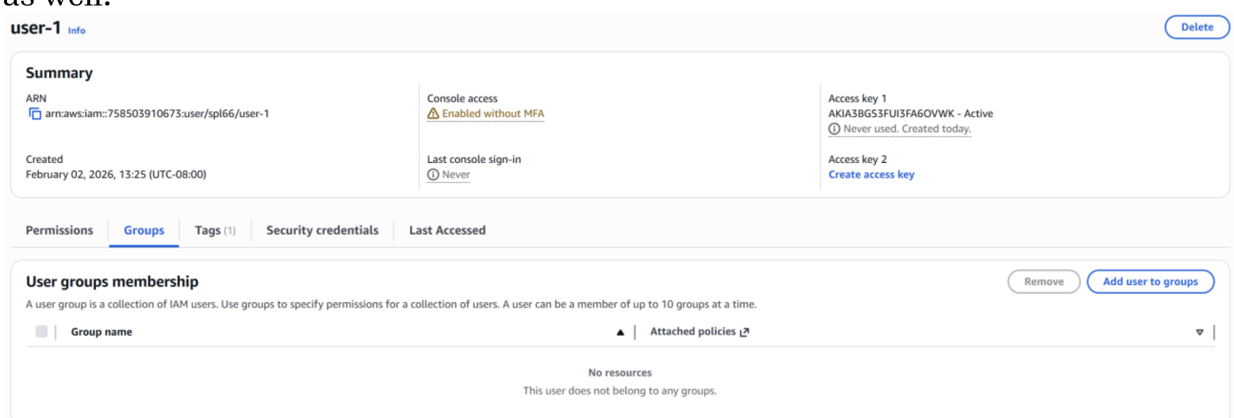

In the left navigation pane, choose Users, notice how there are 3 Users already listed.



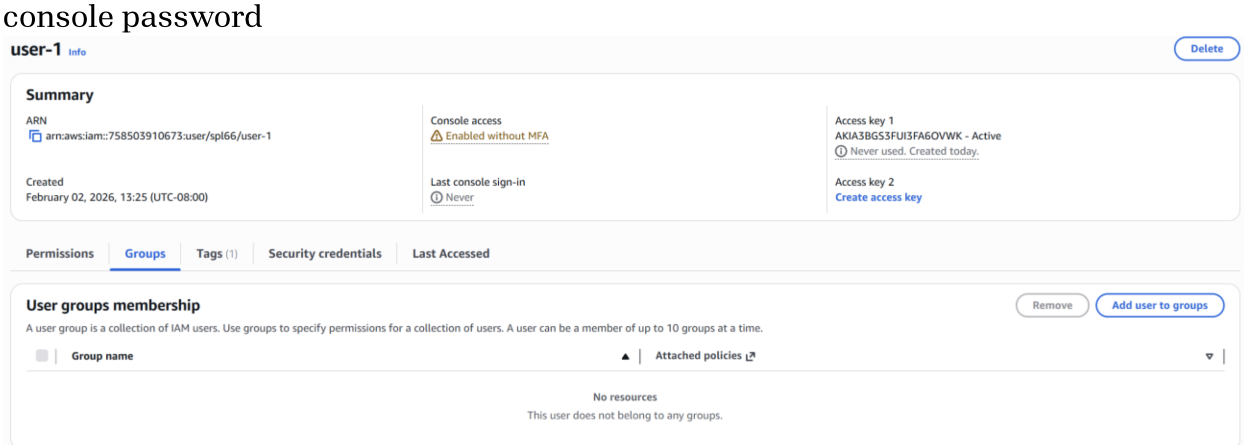

Choose the User-1 Link, notice how there are no polices attached to this user



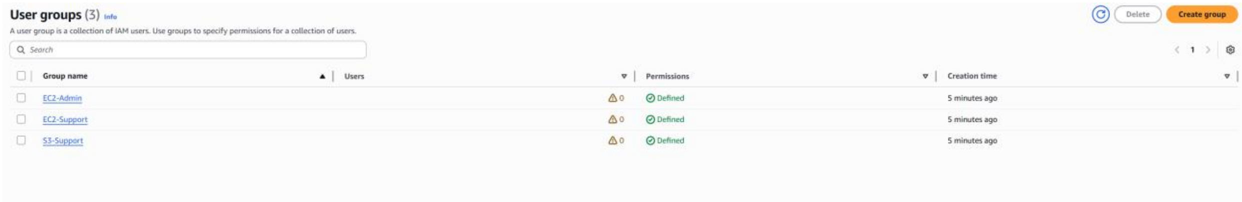

Now navigate to Groups on User-1, Notice how User-1 is not a part of any group as well.



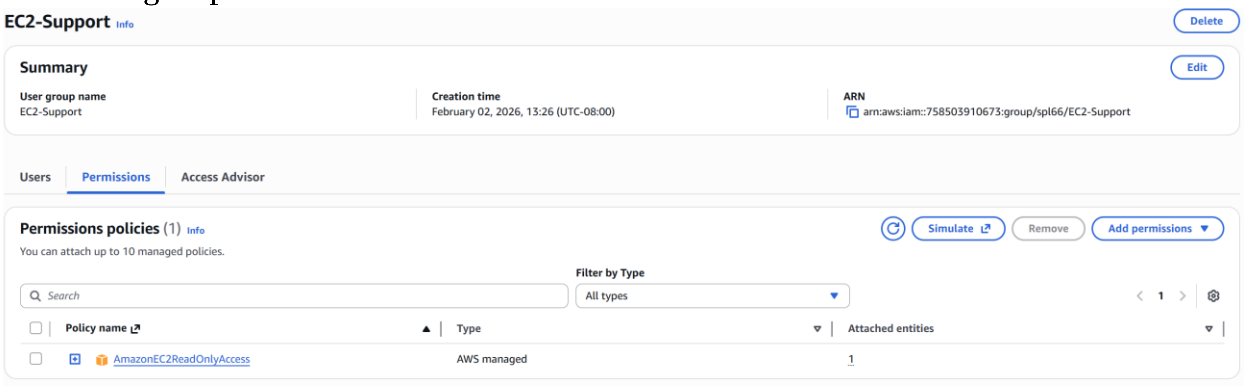

Now Navigate to the Security Credentials Tab and notice how User-1 is assigned a console password




In the left navigation pane choose User Groups and notice the groups that are set up for you.

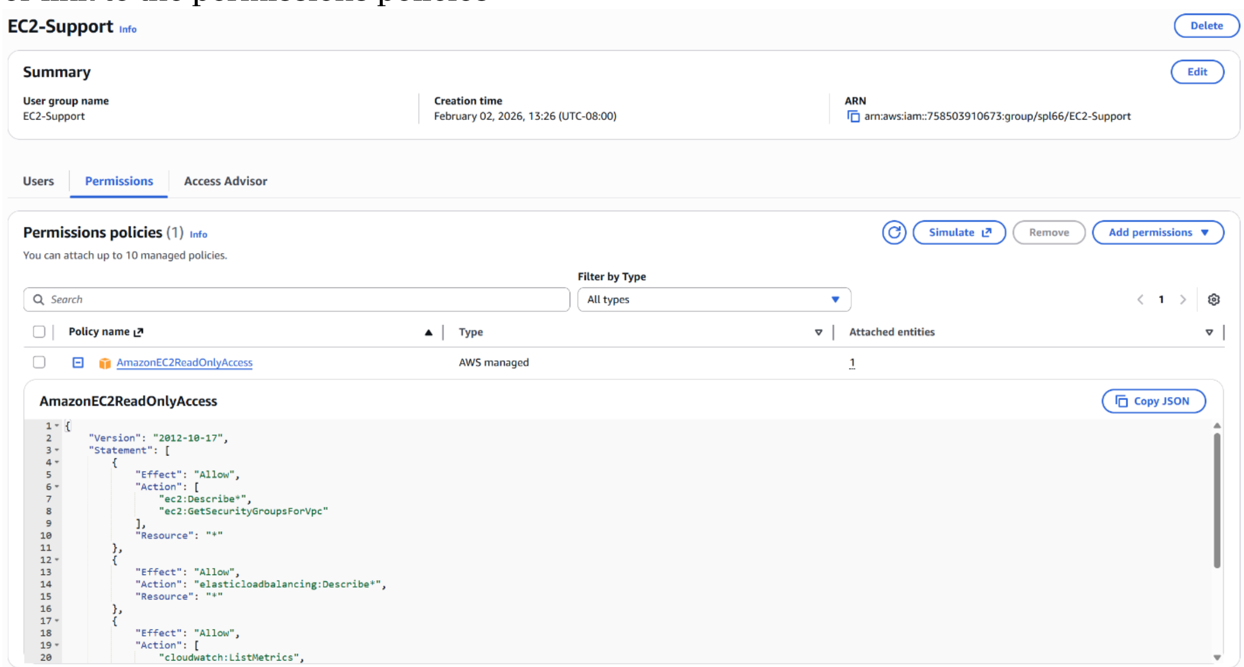



Click each link and go to the permission tab to see what access is allowed for each IAM group

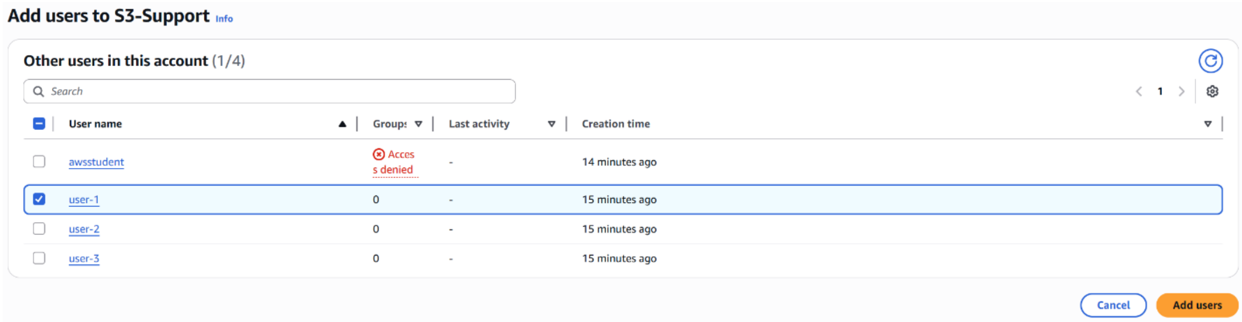



Also notice that no users are in any groups in the users tab

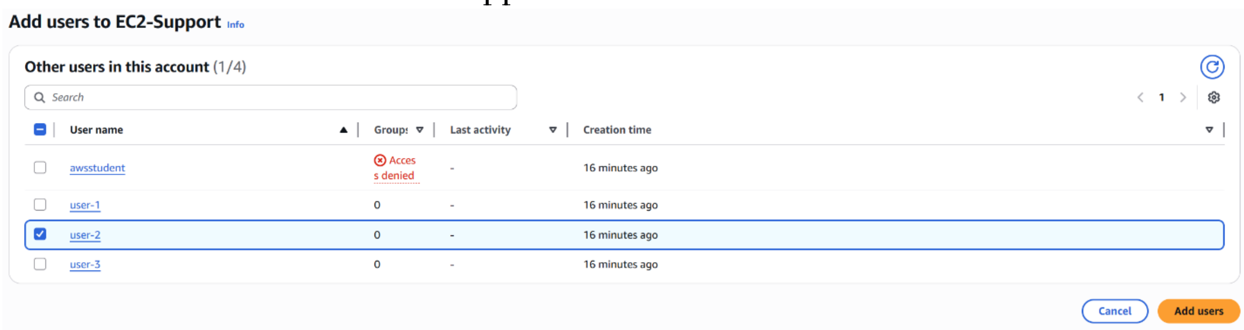
To see how the JSON is structured go to the permissions tab and click the plus or link to the permissions policies



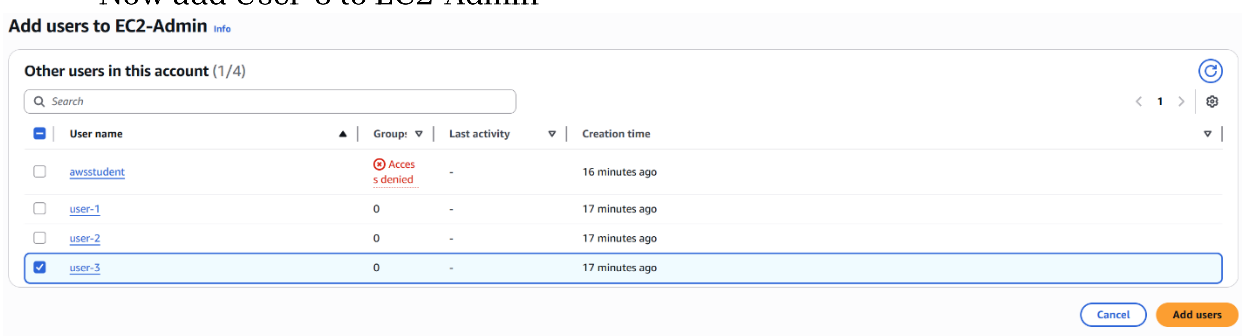
Now add user-1 to the S3-Support group, go to the Users tab and click add users on the top right, and select User-1 to add to S3-Support.



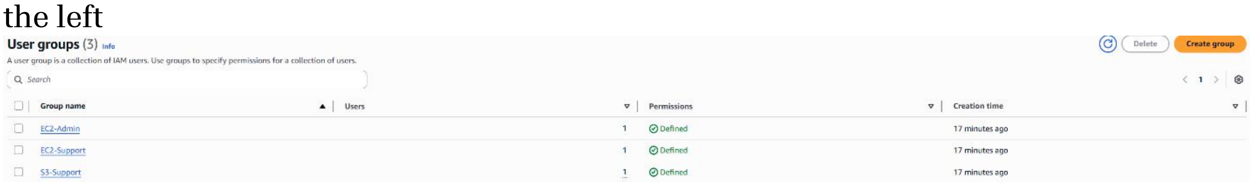

Now add User-2 to EC2-Support



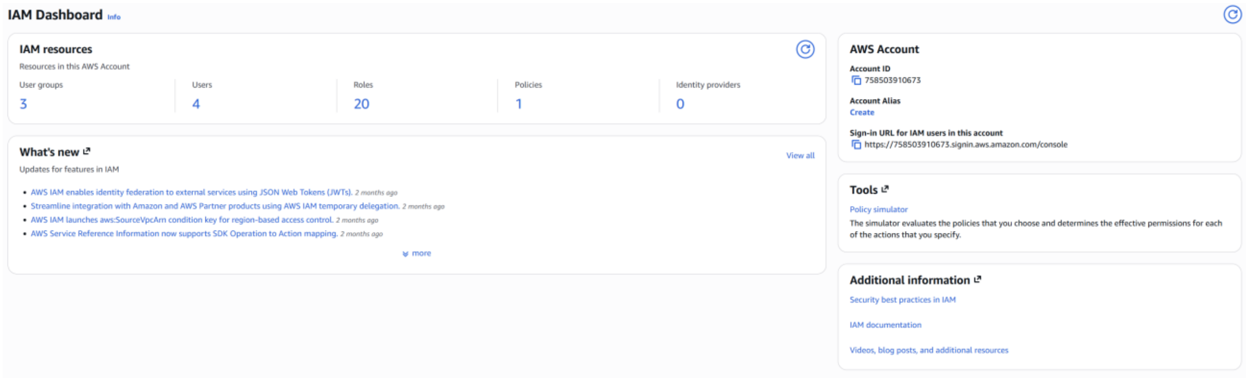

Now add User-3 to EC2-Admin



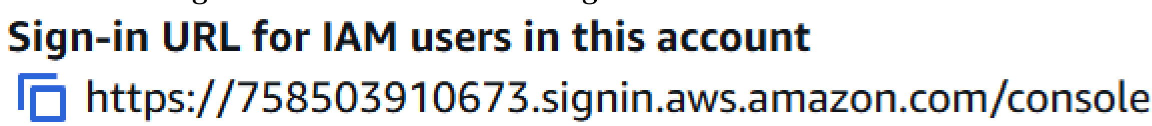

You can verify that there is one user in each group using the User Group tab on the left



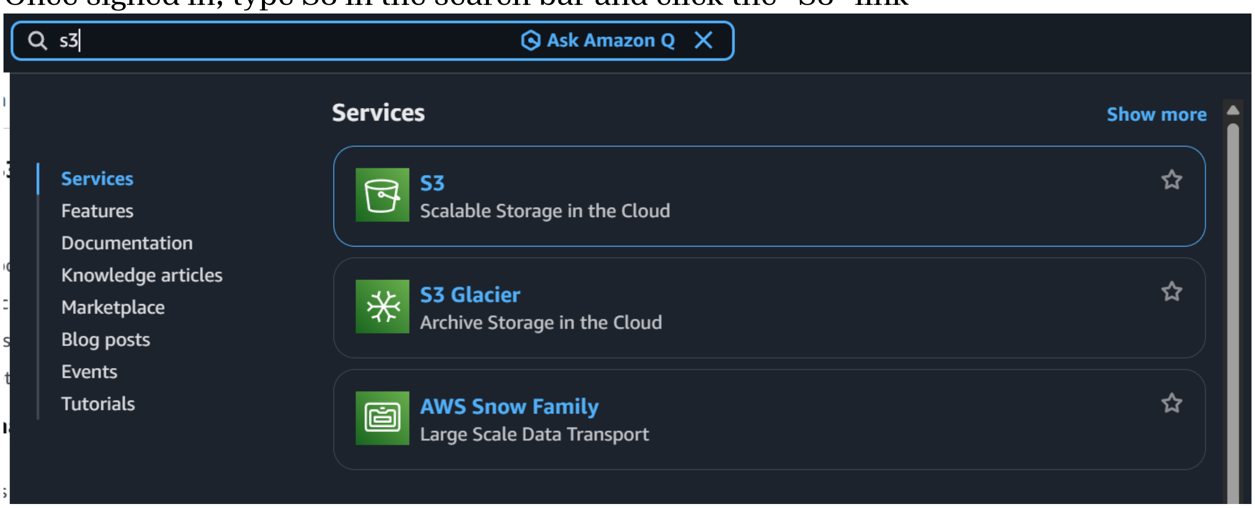
You can also look in the IAM Dashboard



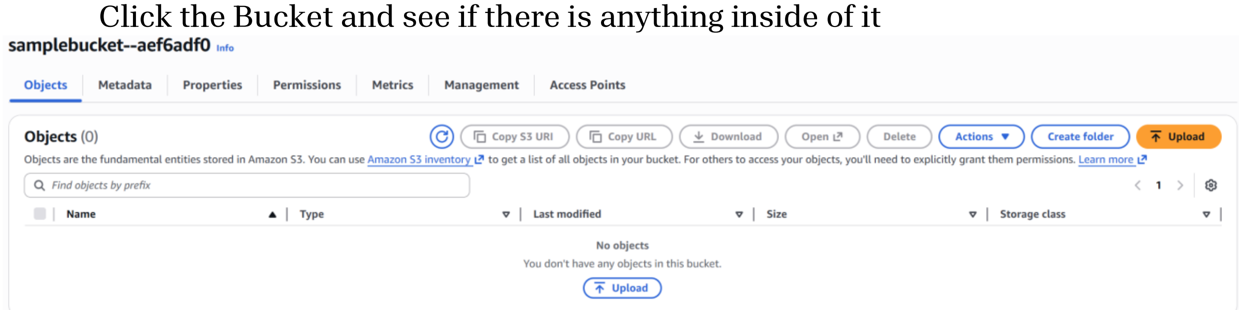

Now Login to one of the Users using the link on the left of teh IAM Dashboard




Copy the URL and open the link in an incognito window or the e1uivelent of your browser.
Now sign-in as User-1 who is assigned to the S3-Support group
IAM username: user-1
Password: Lab-Password1
Once signed in, type S3 in the search bar and click the "S3" link

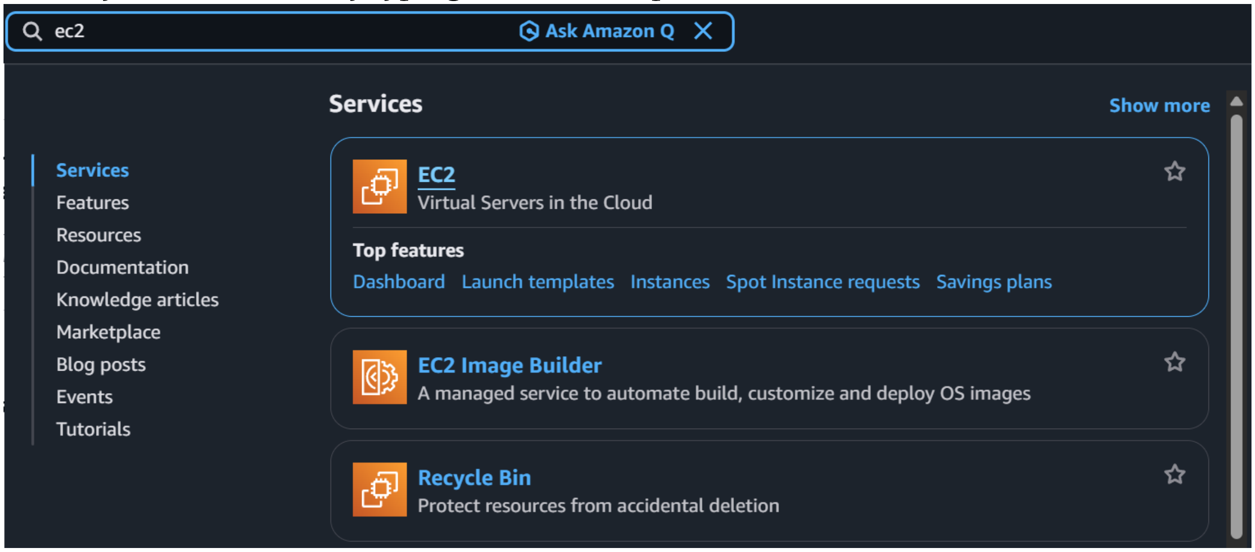



Click the Bucket and see if there is anything inside of it

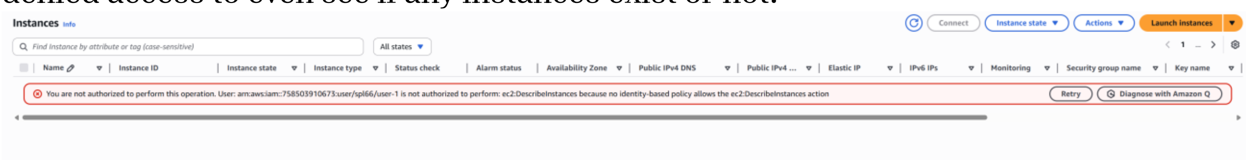



In this image there are no objects in samplebucket—aef6adf0

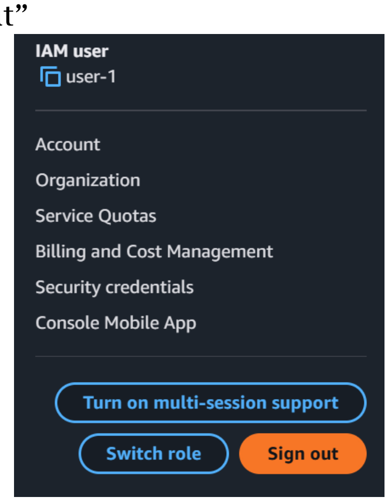
Now try to access EC2 by typing EC2 in the top left search bar




Now navigate to Instances using the left navigation pane and notice how you are denied access to even see if any instances exist or not.




Now sign user-1 out of the console by choosing user-1 on teh top right of the screen and choosing “Sign Out”

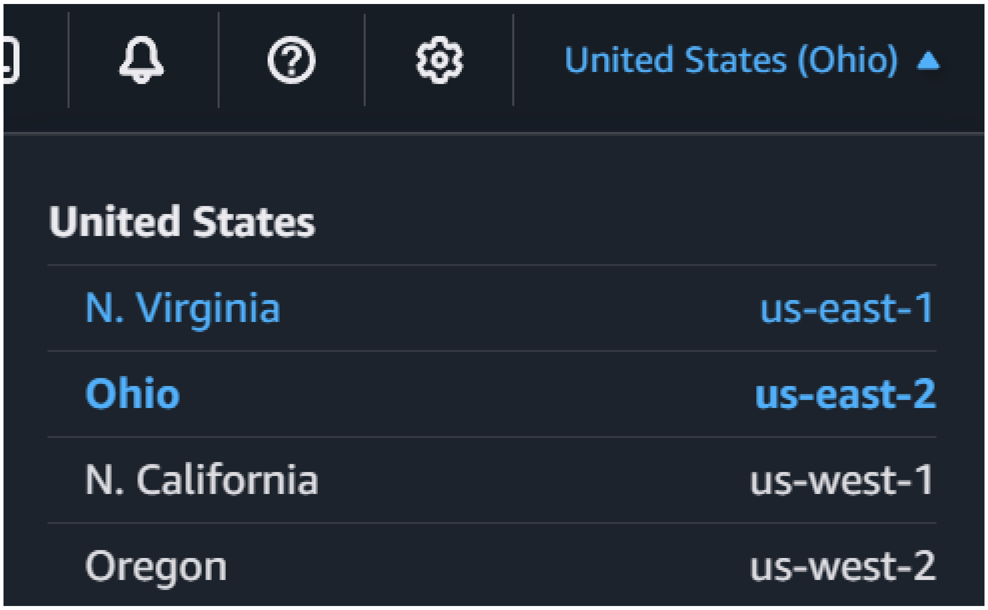



Now repeat the URL pasting process again in an incognito or equivalent window but now sign into user-2

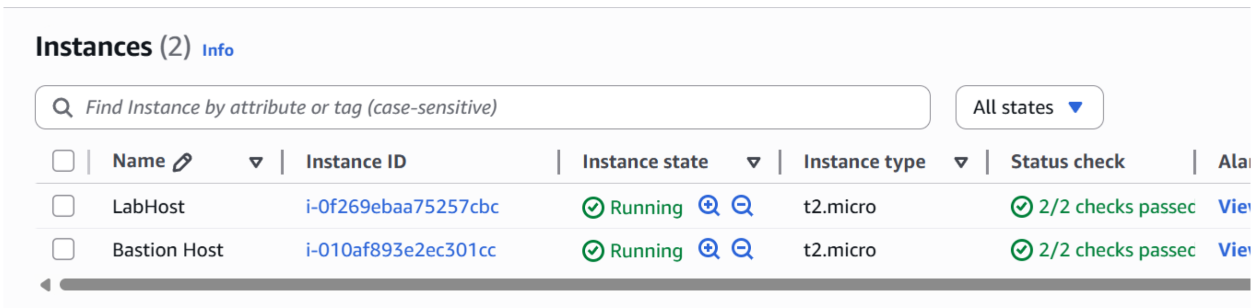
Username: user-2

Password: Lab-Password2

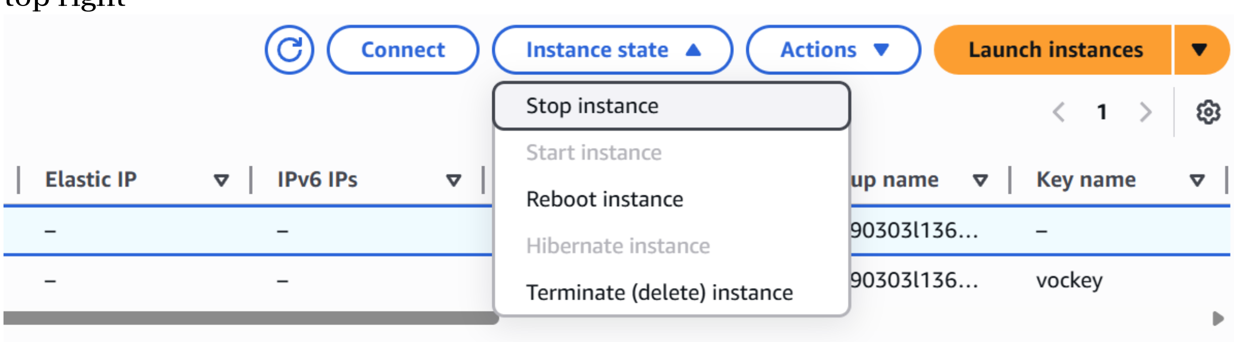
Search EC2 again and navigate to EC2 Instances and change your region in the top right to Virginia if it isn't already




Notice how user-2 can view all instances




Now select the lab host instance and stop it using the instance state menu in the top right



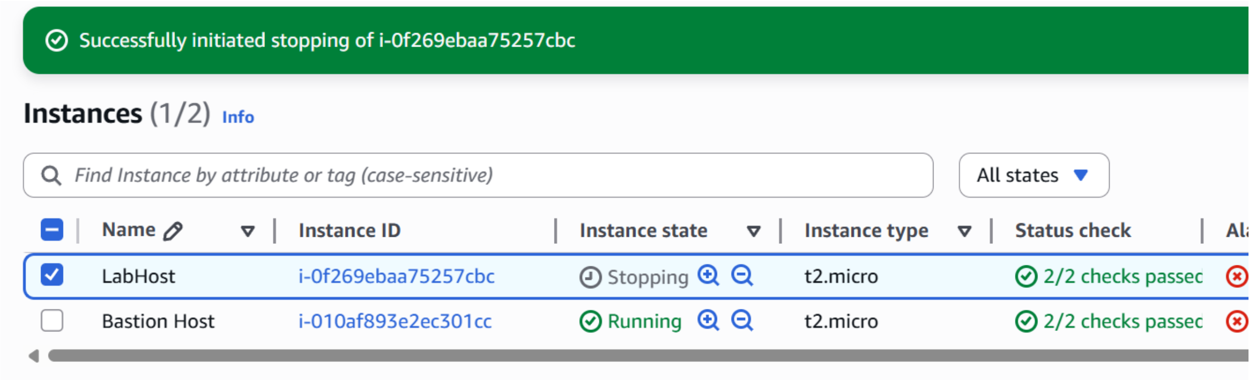

You should receive a failure message as User-2 is not authorized to edit anything but only view EC2 Instances, if you navigate to the S3 console you will see that you do not have permissions to view buckets as well.

Sign out and repeat the same process for stopping the EC2 instance for user-3

Username: user-3

Password: Lab-Password3

This time user-3 will succeed as they have permissions to stop EC2 instances




Now go back to the Vocareuem Workpage submit the lab and get a full score, you can resubmit multiple times until you get a full score.

| Total score | 40/40 |
|---|---|
| TASK 2a - Added user-1 to S3-Support group | 5/5 |
| TASK 2b - Added user-2 to EC2-Support group | 5/5 |
| TASK 2c - Added user-3 to EC2-Admin group | 5/5 |
| TASK 3a - user-1 logged in | 5/5 |
| TASK 3b - user-2 logged in | 5/5 |
| TASK 3c - user-2 ec2 stop instance attempt | 5/5 |
| TASK 3d - user-3 logged in | 5/5 |
| TASK 3e - user-3 EC2 stop instance attempt | 5/5 |

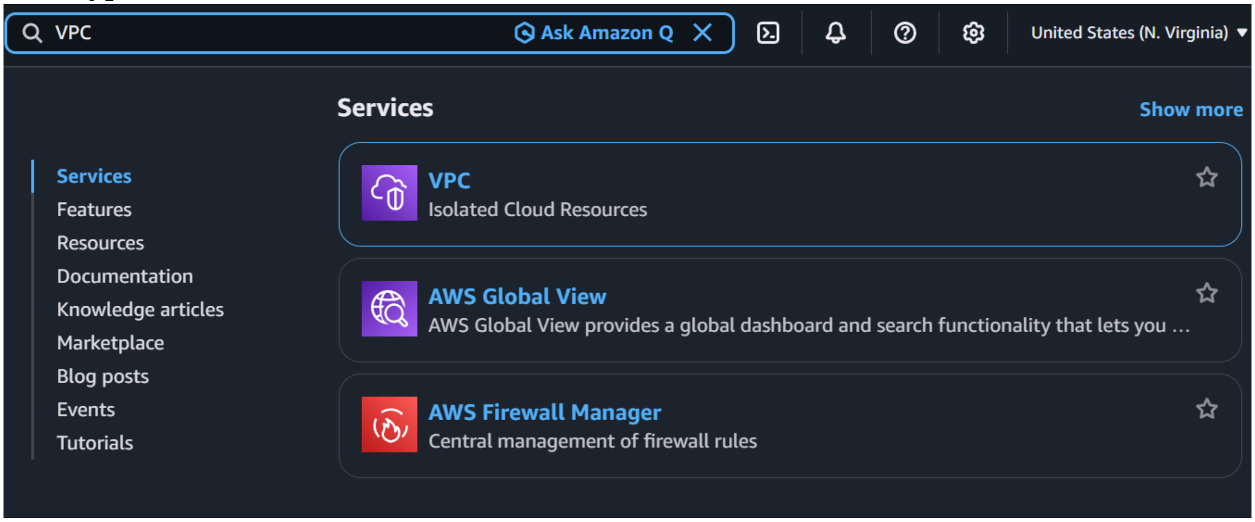
# Lab 2 – Build Your VPC and Launch a Web Server

Repeat the steps to start the lab from Lab 1 and start the lab

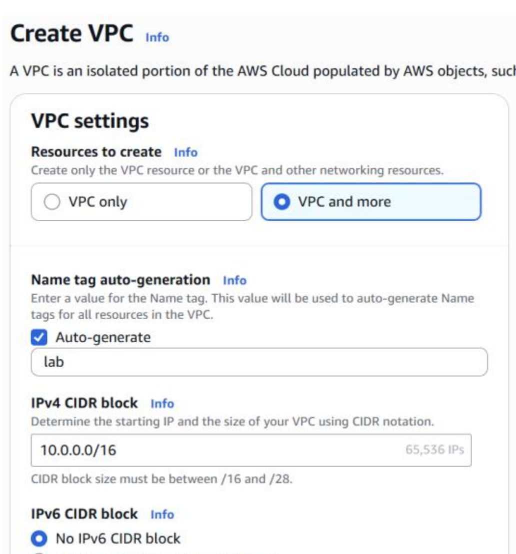
In this lab you will create a VPC with 2 sets of public and private subnets with each set in a different availability zone with a NAT gateway in one public subnet and a web server in the other public subnet.

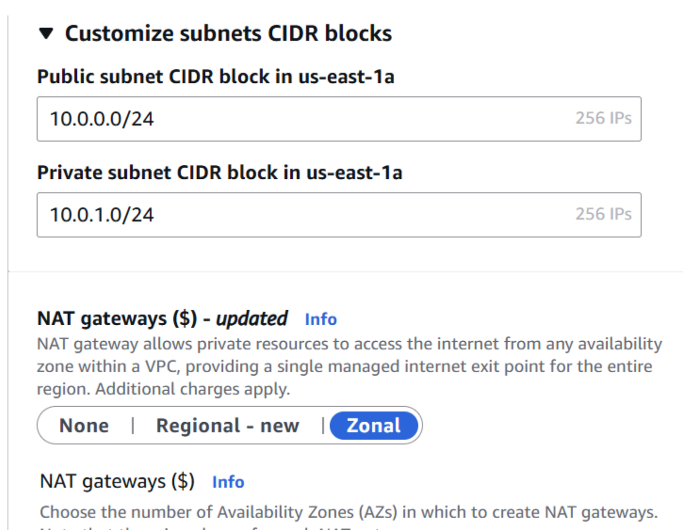
Now start by verifying that you are in the region N. Virginia (us-east-1)


Now type VPC in the console search bar and click VPC




Now Choose Create VPC and configure it with eh following settings
Choose: VPC and More
Under Name tag auto-generation, keep auto-generate selection but change the name form project to lab
Keep the IPv4 CIDR Block to 10.0.0.0/16 these are going to be the IP addresses assigned to the resources in the subnets
For number of AZs or Availability zones to 1
For number of public and private subnets choose 1
Expand the Customize subnets CIDR blocks section
Change the public subnet CIDR block to 10.0.0.0/24
Change the private subnet CIDR block to 10.0.1.0/24
Set the NAT gateways to In 1 AZ
Set VPC endpoint to none
Keep both DNS hostnames and DNS resolution enabled



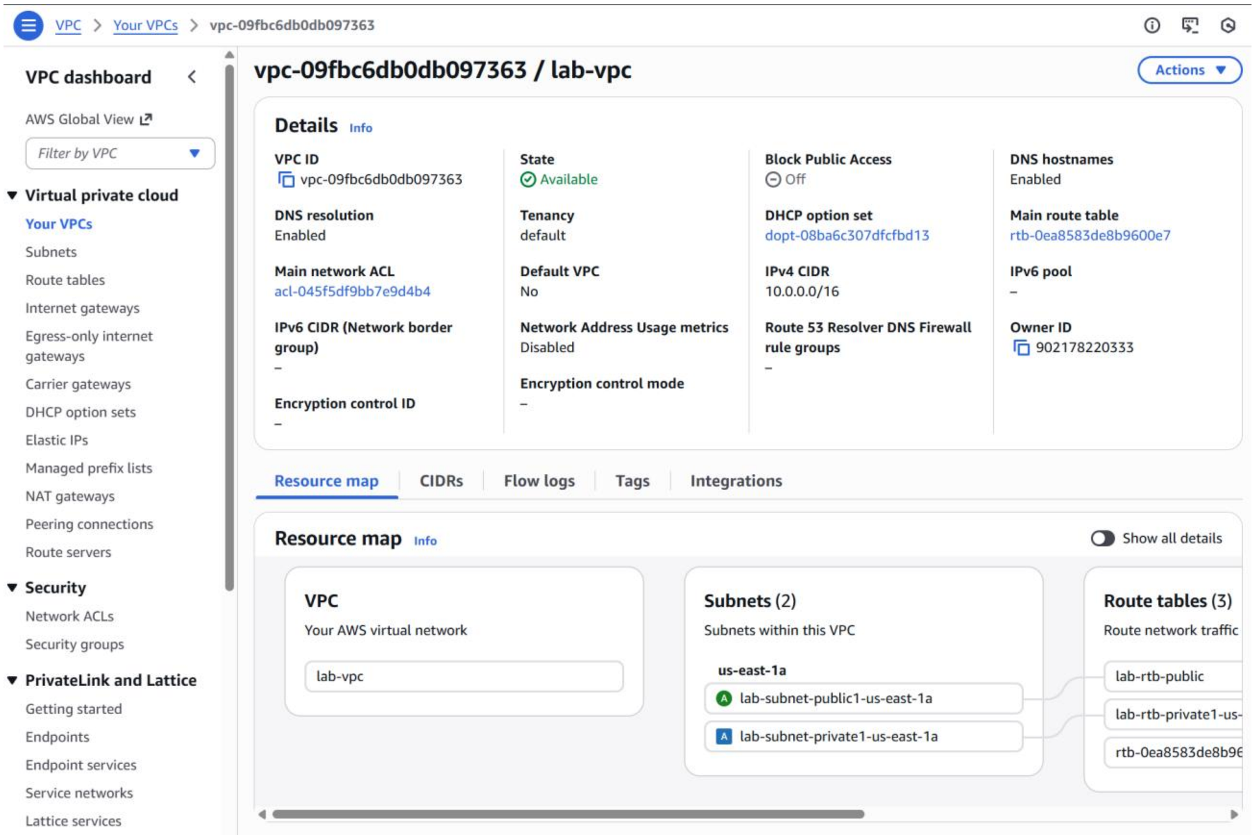



In the right-side preview panel verify the configurations to the Lab VPC

VPC: lab-vpc

- Subnets:
  - us-east-1a
    - Public subnet name: lab-subnet-public1-us-east-1a
    - Private subnet name: lab-subnet-private1-us-east-1a
- Route tables
  - lab-rtb-public
  - lab-rtb-private1-us-east-1a
- Network connections
  - lab-igw
  - lab-nat-public1-us-east-1a

Now Choose Create VPC and then View VPC




Now Create the second set of subnets; in the left navigation pane choose subnets

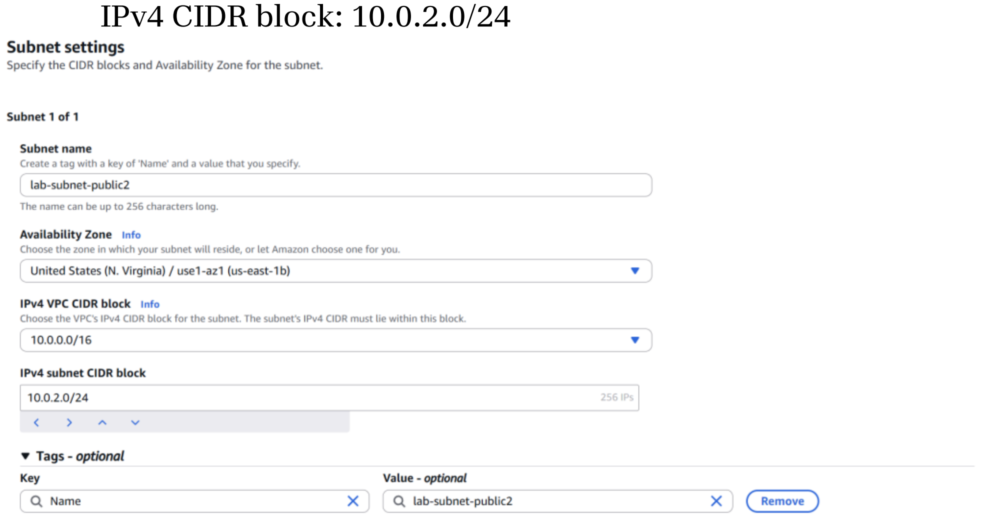
Choose Create Subnets and configure:

VPC ID: lab-vpc (select from the menu).

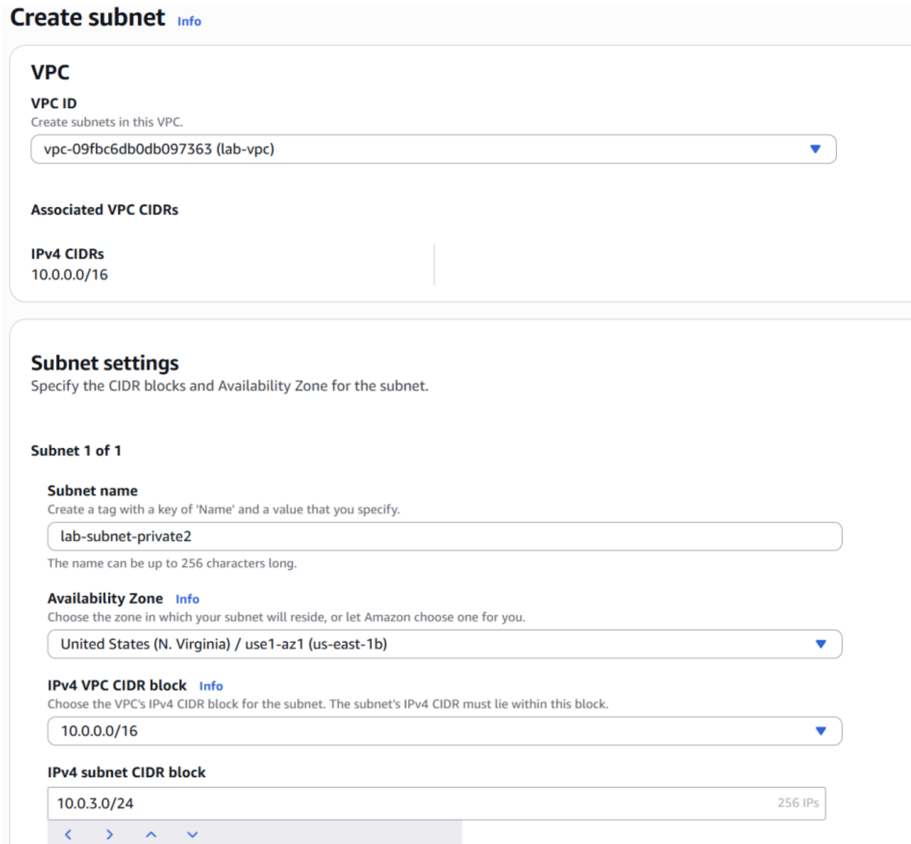
Subnet name: lab-subnet-public2

Availability Zone: Select the second Availability Zone (ex: us-east-1b)

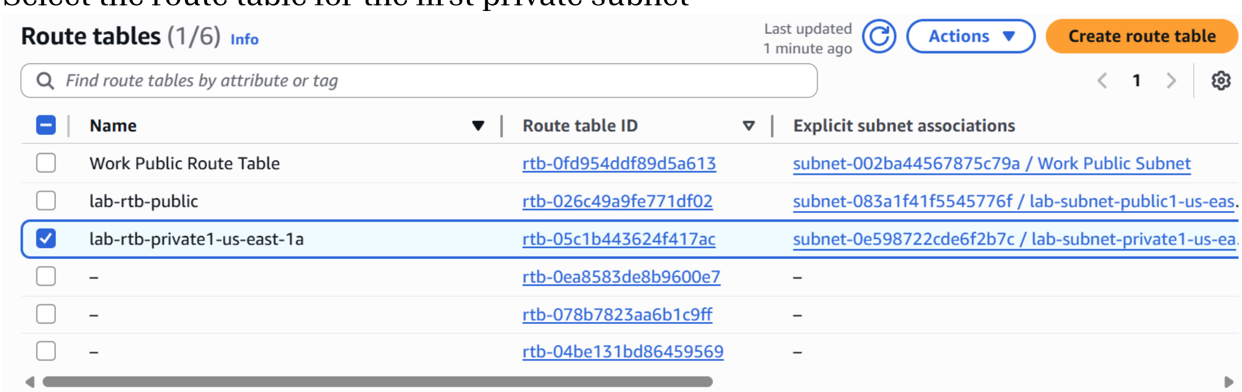
IPv4 CIDR block: 10.0.2.0/24



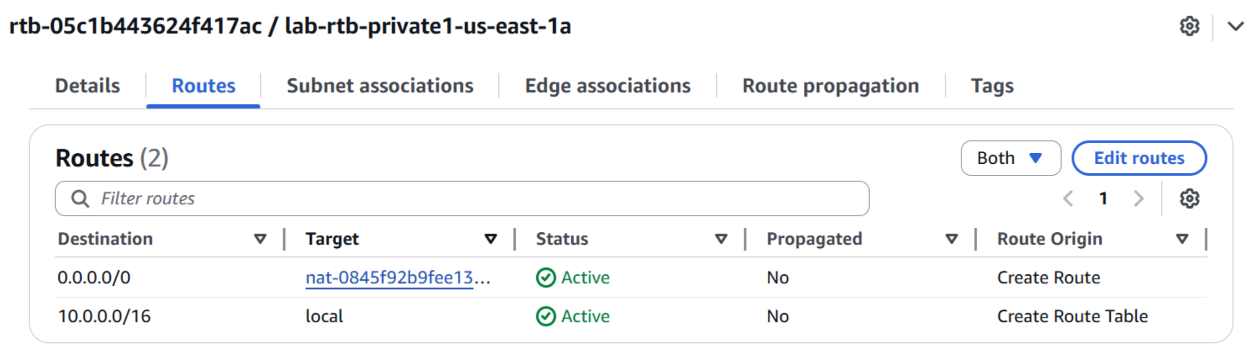

Create the Subnet; Repeat the process but change the name from “lab-subnet-public2" to “lab-subnet-private2" and change the IPv6 CIDR block to 10.0.3.0/24




Now that the 2 subnets have been created, navigate to the Route Tables using the left navigation pane.
Select the route table for the first private subnet




Go to the routes tab and notice the 0.0.0.0/0 route point to the NAT gateway this means all unresolved traffic will go to the internet.



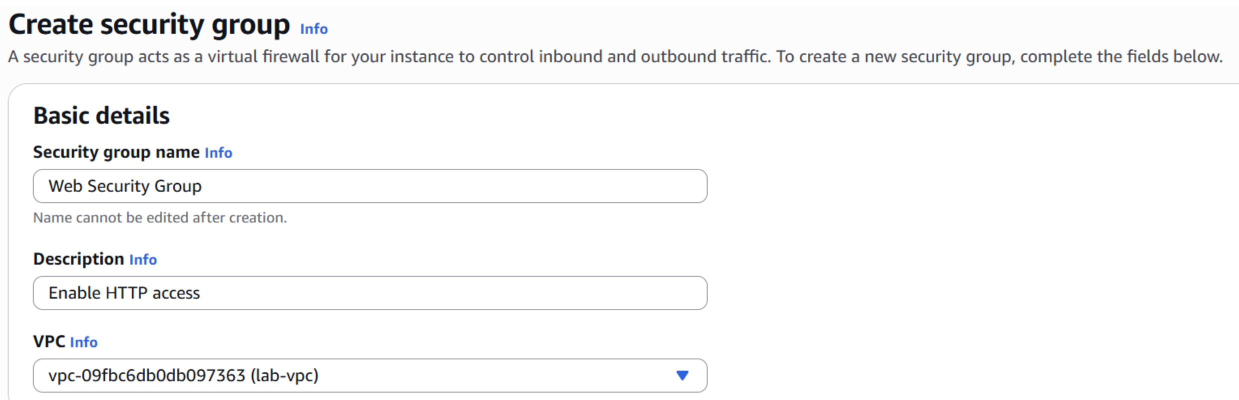

Now go to the Subnet associations tab and notice how it is associated with only itself, but now we want to associate itself with the second private subnet as well. Choose edit Subnet Associations and select the lab-subenet-private2


Choose Save Associations and repeat this process for the public subnets

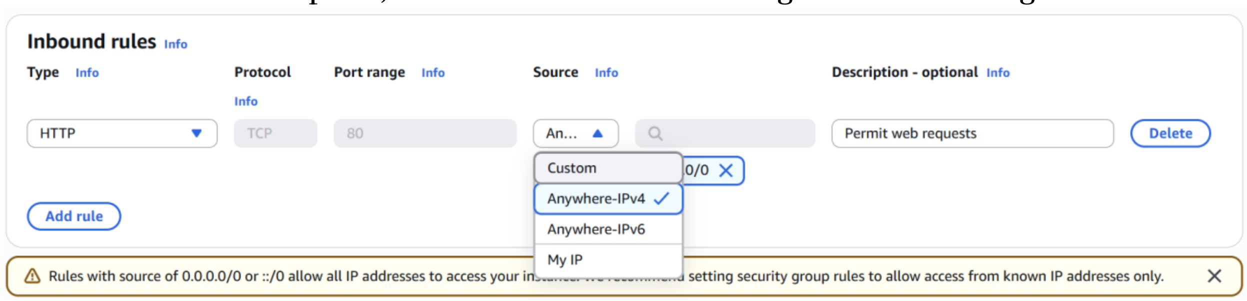

Next you will create a VPC security group which acts as a virtual firewall. When you launch an instance, you associate one or more security groups with the instance. You can add rules to each security group that allow traffic to or from its associated instances.

In the left navigation pane choose Security Groups

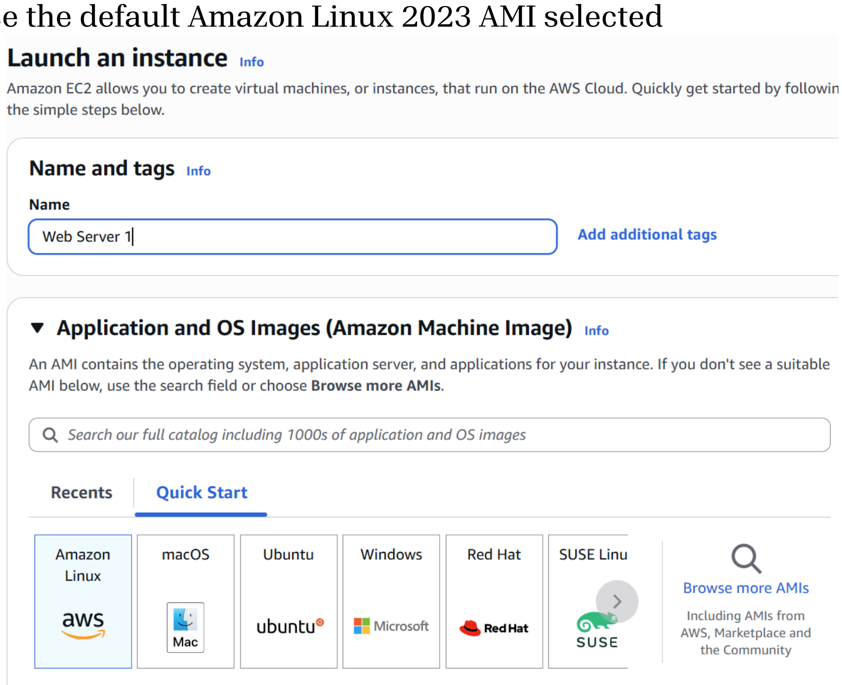
Choose Create Security Group and configure the following



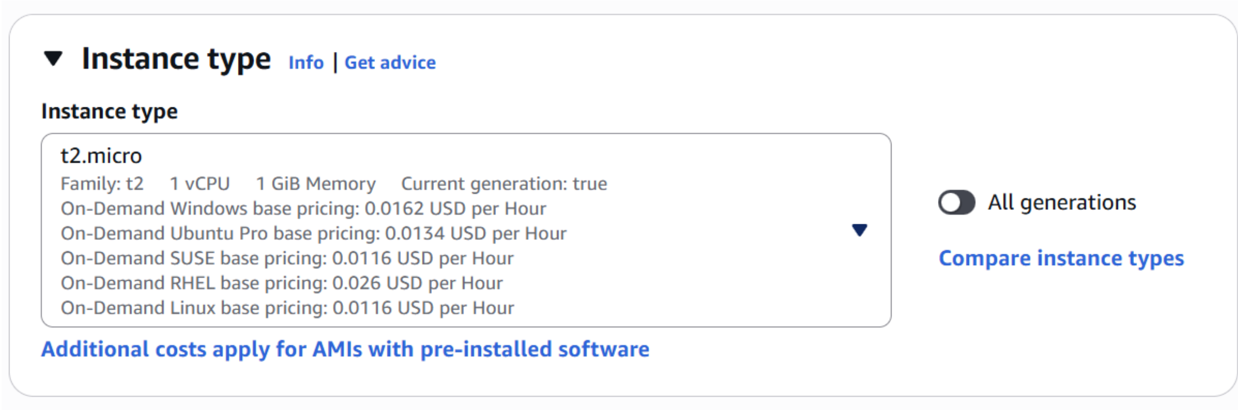

In the Inbound Rules pane, choose Add Rule and configure the following



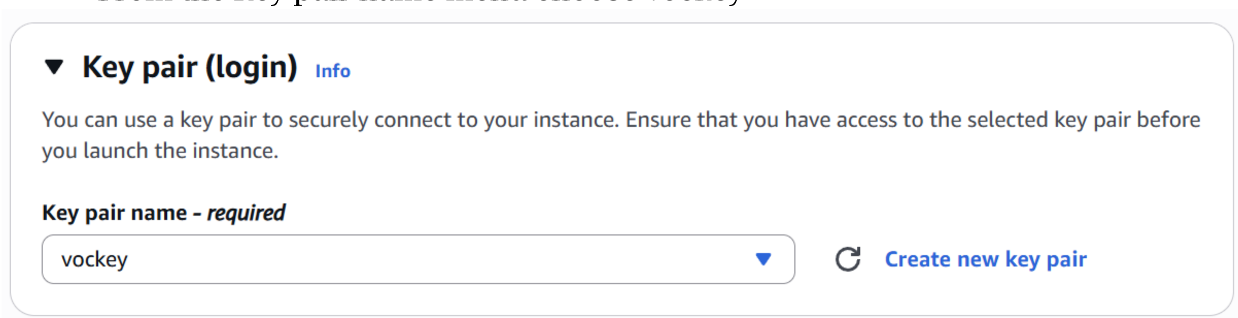

Scroll down and choose Create Security Group
Now you will launch a Web server Instance
Use the console search bar and search EC2 and click on EC2
From the Launch Instance menu choose Launch instance
Name the instance: Web Server 1
Choose the default Amazon Linux 2023 AMI selected




Then in the Instance Type Choose t2.micro




From the Key pair name menu choose vockey




Configure the Network Settings, next to the Netowrk settings Choose Edit and configure:

Network: lab-vpc

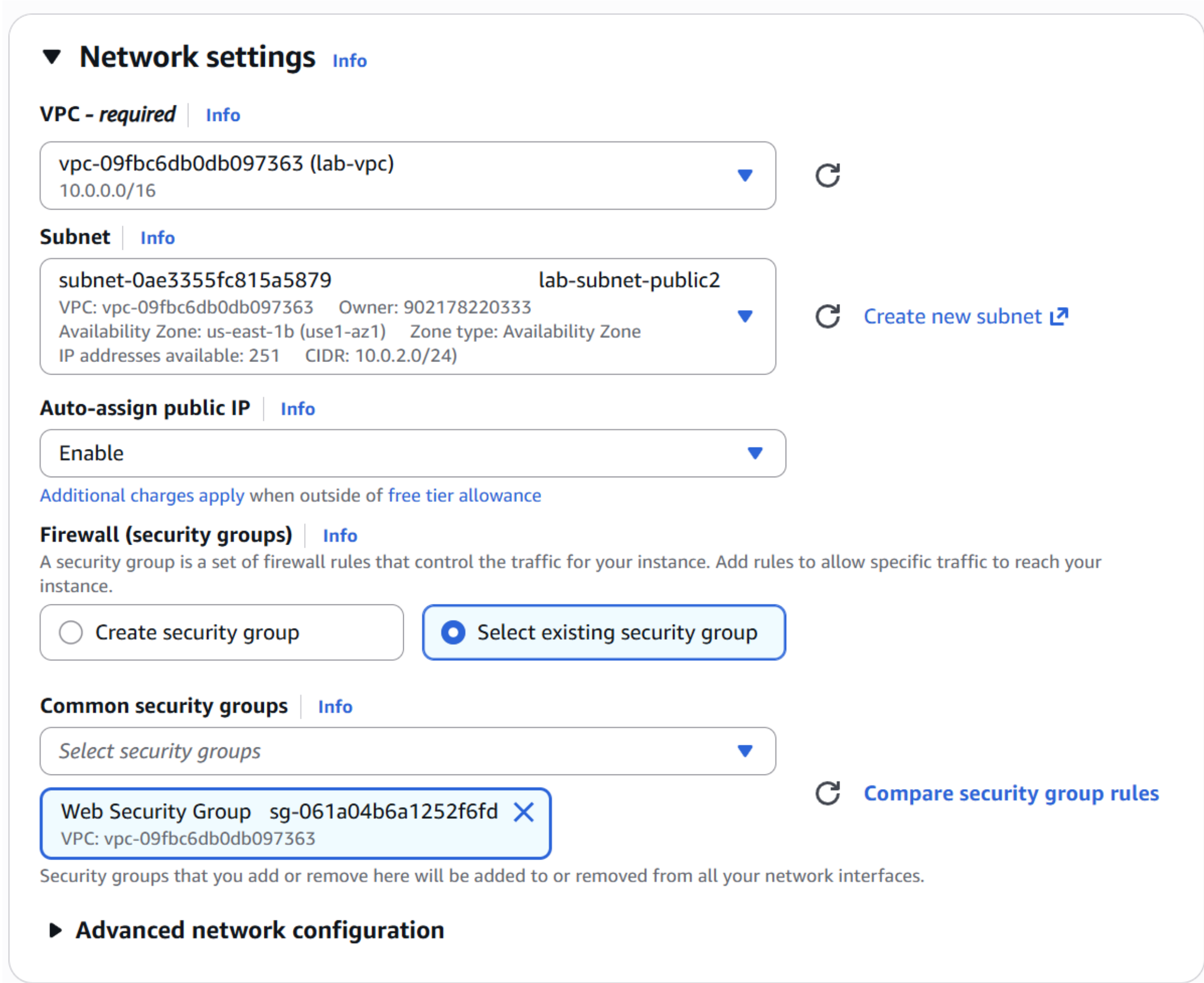
Subnet: lab-subnet-public2 (not Private!)

Auto-assign public IP: Enable

Next Assign it the security group created earlier

Under Firewall (security groups), choose Select existing security groups.

For Common Security Groups, select Web Security Group.

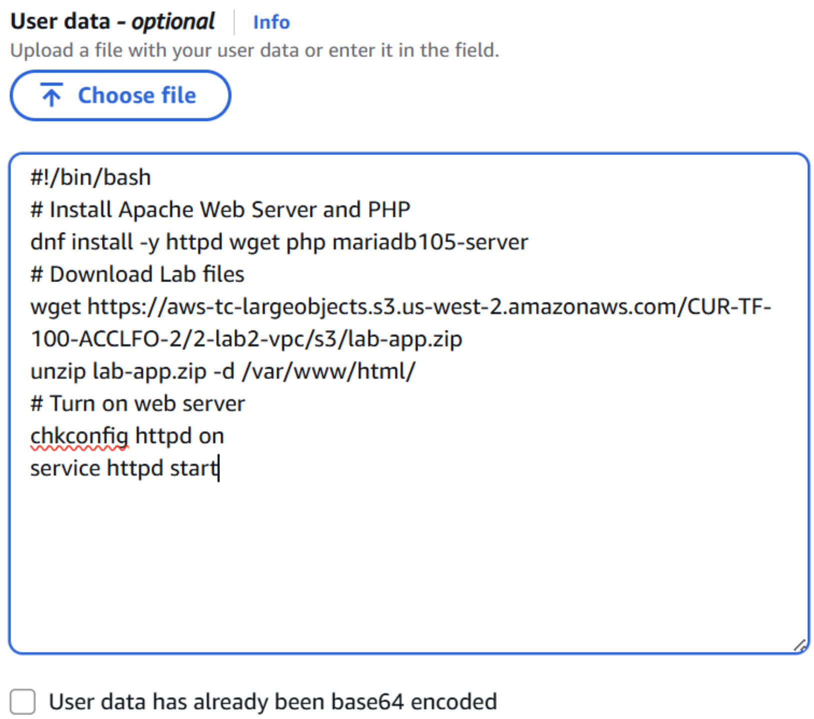



In the configure storage setting keep everything to its defaut settings

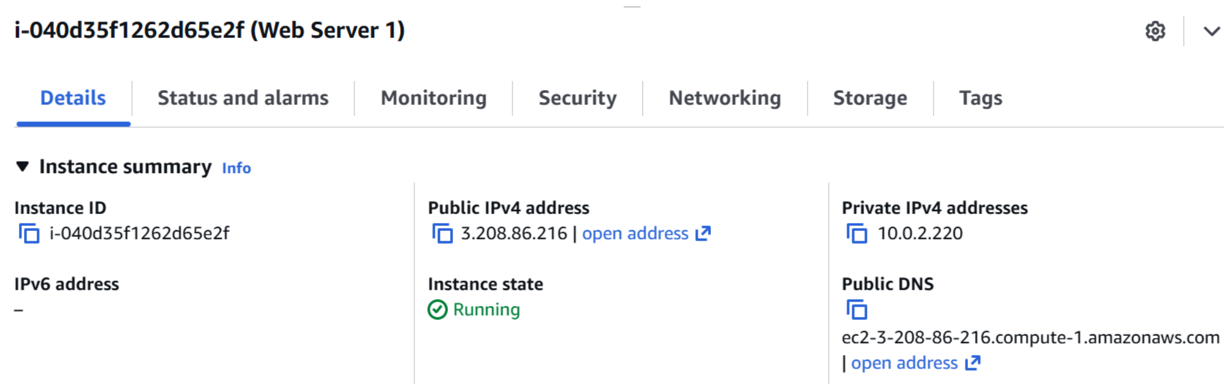
Configure a script to run on the instance when it launches:

Expand the Advanced Details Panel and scroll to the bottom of the page and copy paste the following code:

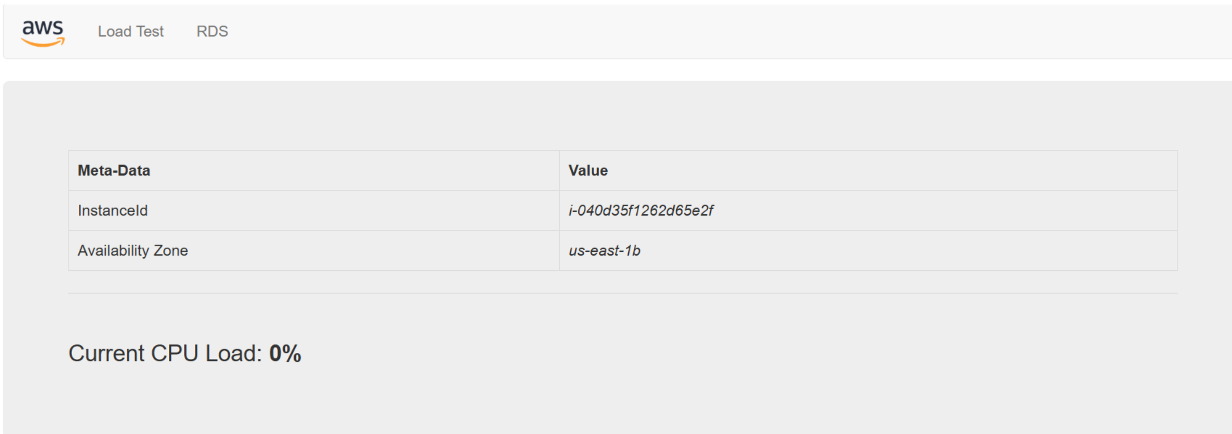
```
#!/bin/bash
# Install Apache Web Server and PHP
dnf install -y httpd wget php mariadb105-server
# Download Lab files
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-ACCLFO-2/2-lab2-vpc/s3/lab-app.zip
unzip lab-app.zip -d /var/www/html/
# Turn on web server
chkconfig httpd on
service httpd start
```



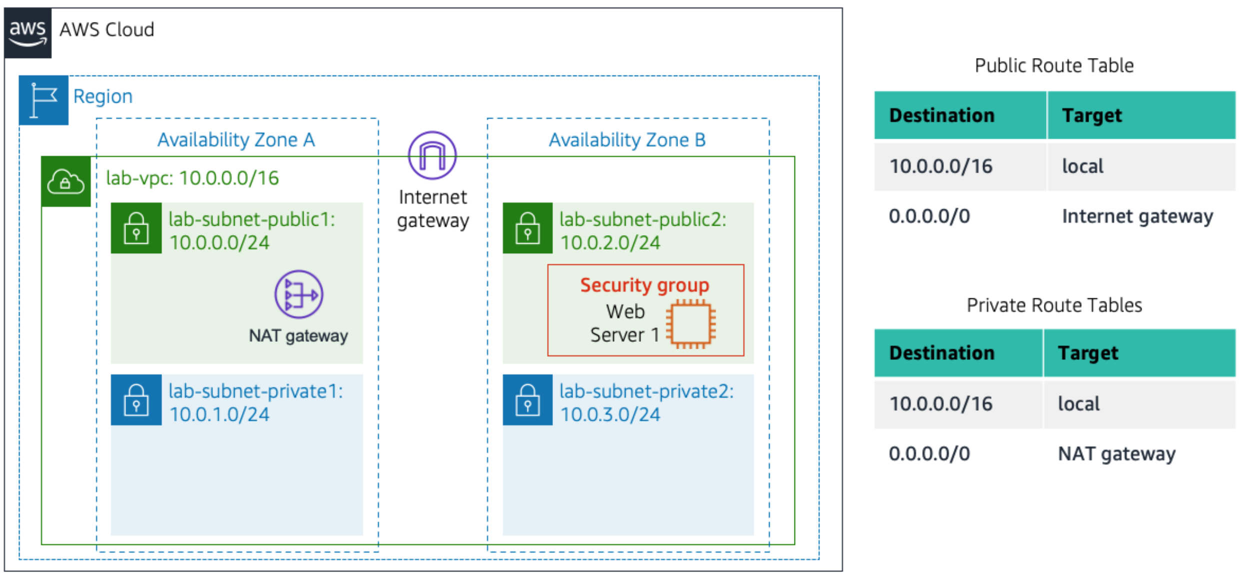

Now Choose Launch Instance, then choose to view all instances and wait until Web Server 1 shows 2/2 checks passed, then select the web server and copy the public DNS value



Open the Link in a new tab. You should see a web page displaying the AWS logo and instance meta-data values.




The complete architecture deployed is:



Now Submit the work for a full score:

| Total score | 30/30 |
| --- | --- |
| Task 1 - VPC created correctly | 5/5 |
| Task 2a - New subnets created correctly | 5/5 |
| Task 2b - Subnet route table association | 5/5 |
| Task 3 - Security group created correctly | 5/5 |
| Task 4a - EC2 instance created correctly | 5/5 |
| Task 4b - EC2 instance website accessible | 5/5 |

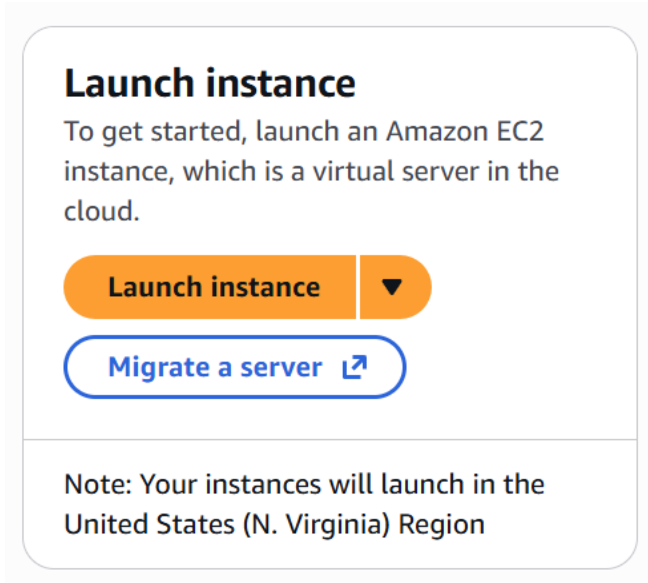
# Lab 3 – Introduction to Amazon EC2

In this lab you will be taught the basic overview of launching, resizing, managing and monitoring an Amazon EC2 Instance

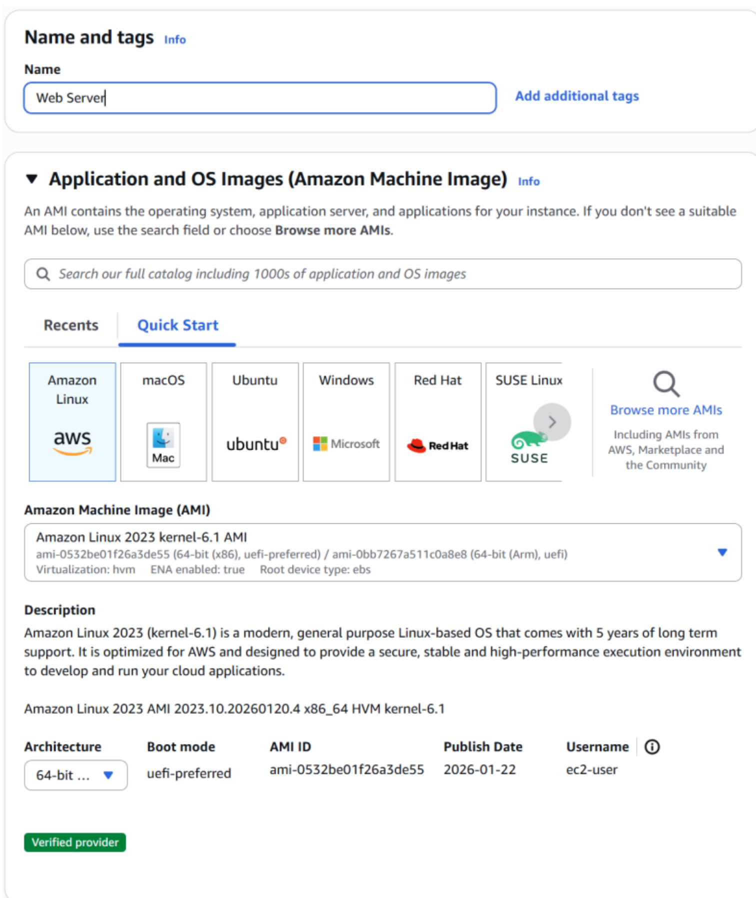
Launch the Lab using the steps from Lab 1 instructions and make sure your region is set to N. Virginia (us-east-1) using the steps covered in Lab 1.

Now Navigate to EC2 using the search bar on the top left of the console. Double check between every step if you are in the correct region.

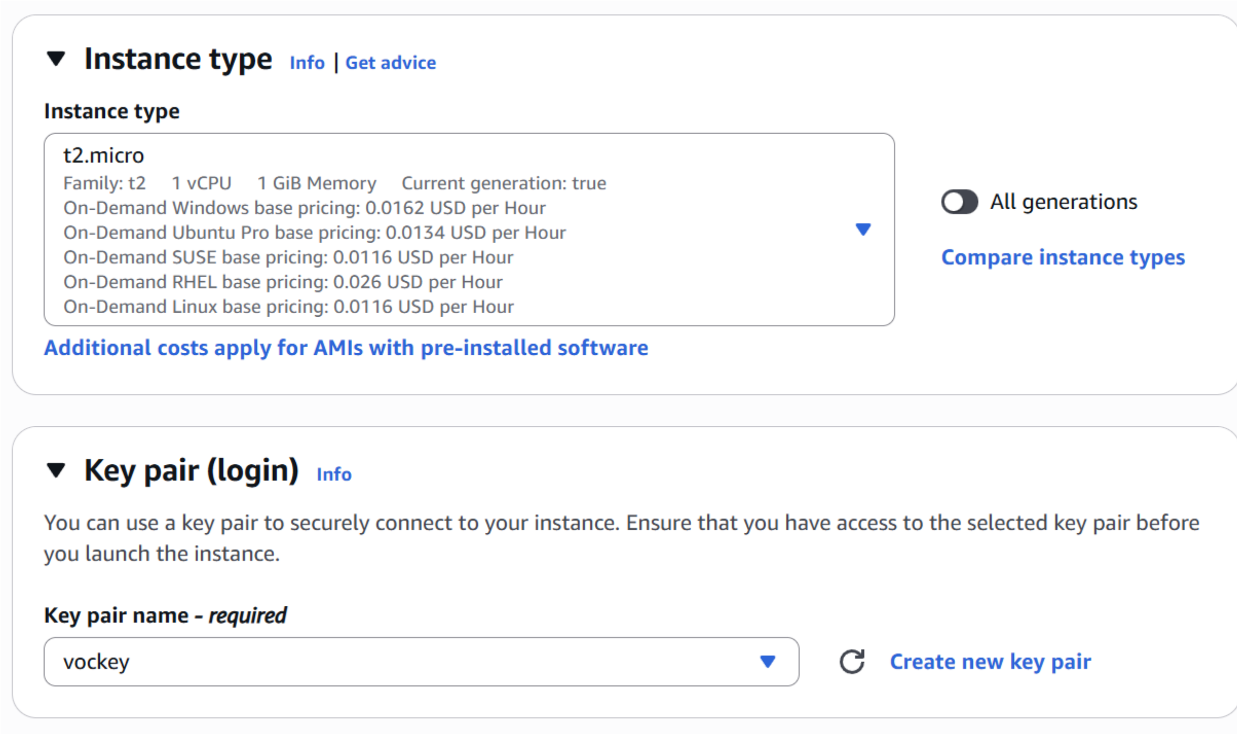
Choose the Launch Instance Menu and Choose Launch Instance.




Name the Instance: Web Server
This name will be stored as a tag with a key of “name” and value of “Web Server”
In the quick start list choose Amazon Linux




In the instance type choose t2.micro
For the Key pair (login) name it “vockey”




Go into the Network Setting by pressing Edit

Choose the pre-made VPC for you (Lab VPC)

Also choose the created subnet: PublicSubnet1

Under Firewall (security groups create a new security group, this will determine what the EC2 Instance can access and what can access the EC2 Instance.

For this Security group we will remove all inbound security groups to prevent all access as security groups only specify allowed connections and will deny anything not allowed.

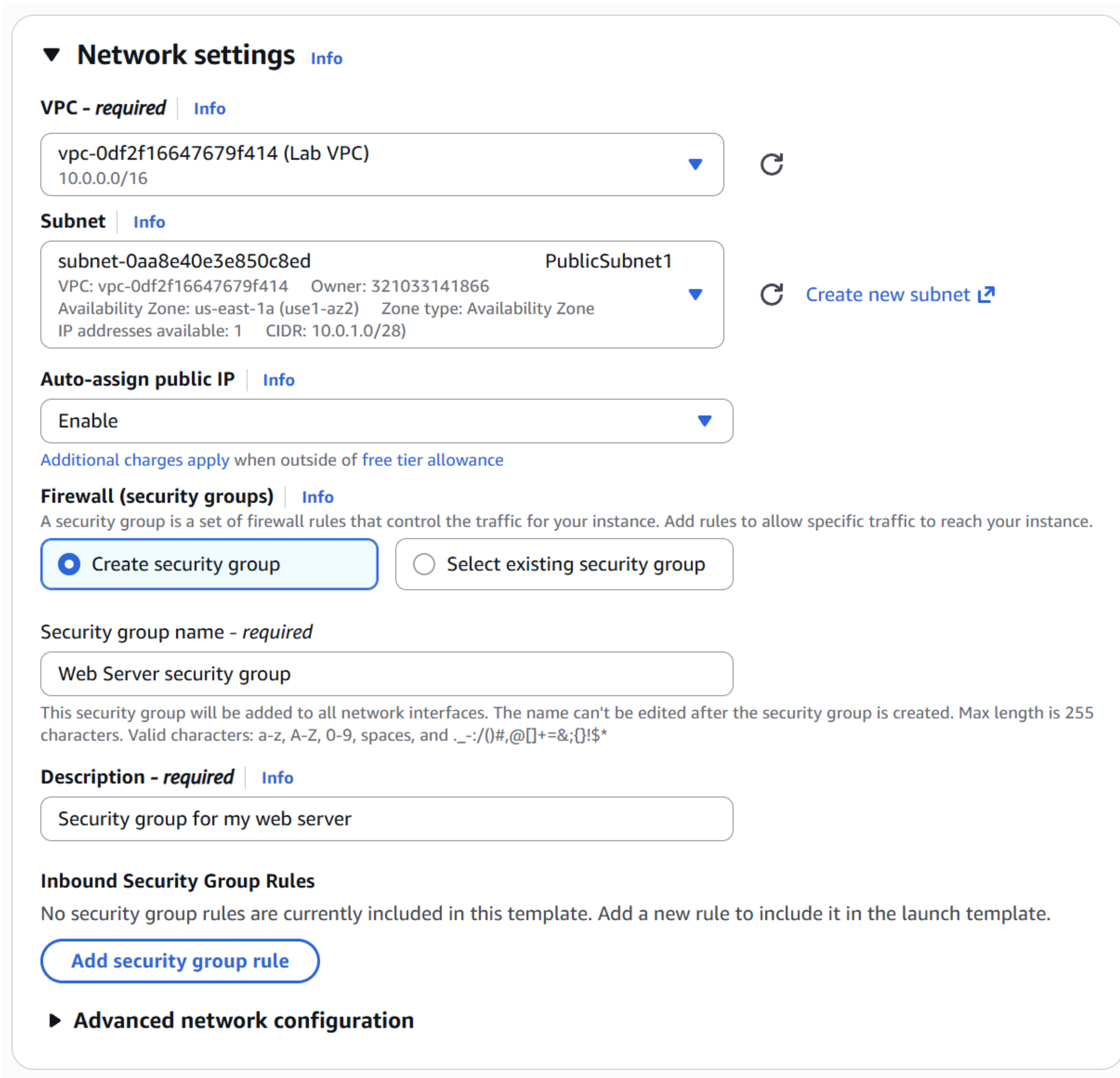
Configure the following for the Security Group:

Name: Web Server security group

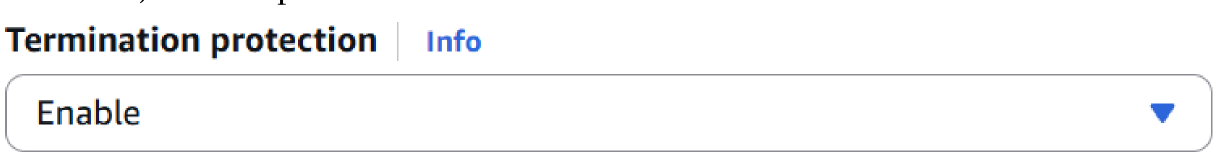
Description: Security group for my web server

Remove the default inbound rule

Keep the default storage settings. The default is 8GB of disk volume and is known as the root volume. These disks are network-attached virtual disks called the Elastic block store.




Towards the bottom, expand the Advanced details. Enable Termination protection, this will prevent accidental termination of an instance.

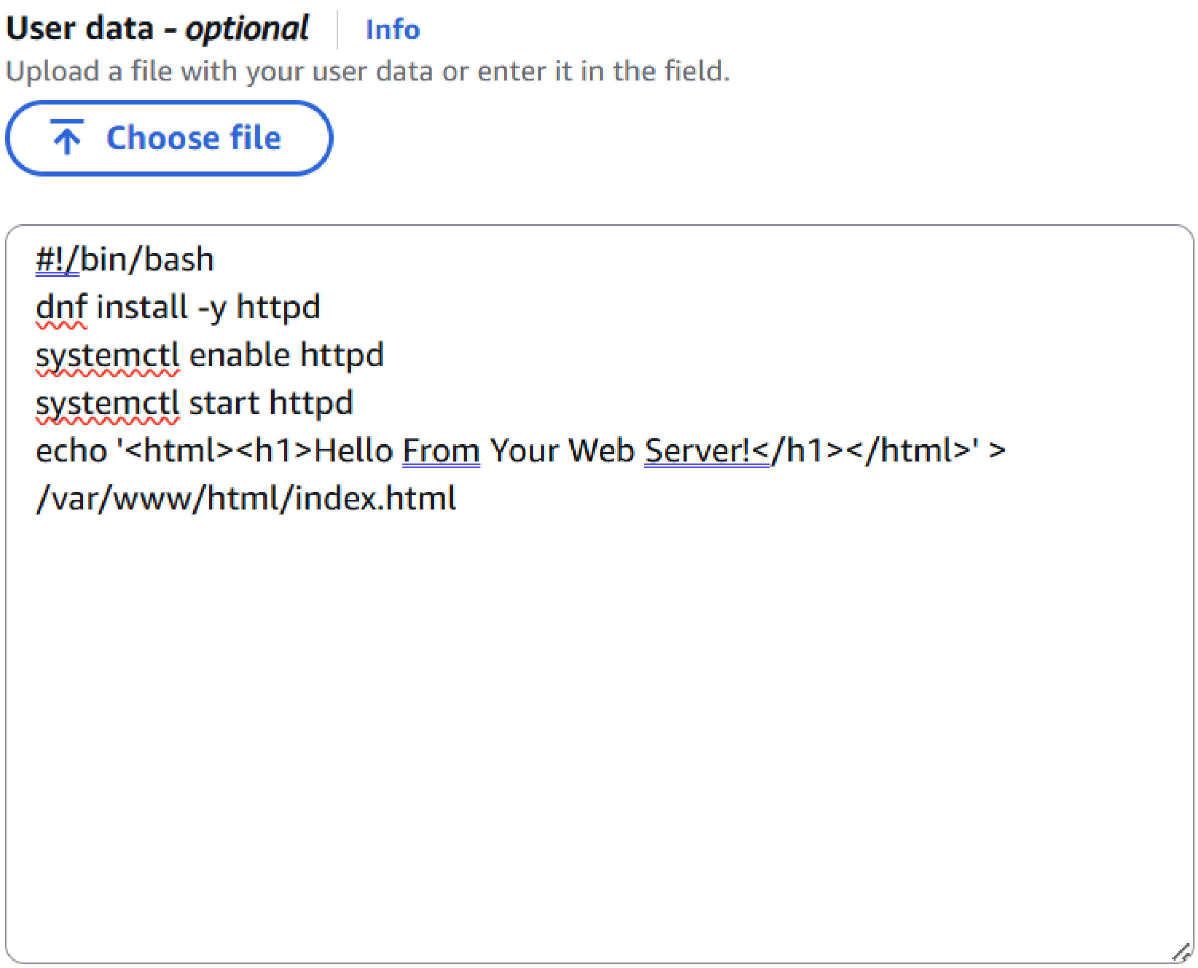



Scroll to the bottom now and copy paste the code below into the User Data box:

```
#!/bin/bash
dnf install -y httpd
systemctl enable httpd
systemctl start httpd
echo '<html><h1>Hello From Your Web Server!</h1></html>' > /var/www/html/index.html
```

This code will automatically run when the instance is launched to automate tasks. This script will do the following:

- Install an Apache web server (httpd)
- Configure the web server to automatically start on boot
- Run the Web server once it has finished installing
- Create a simple web page

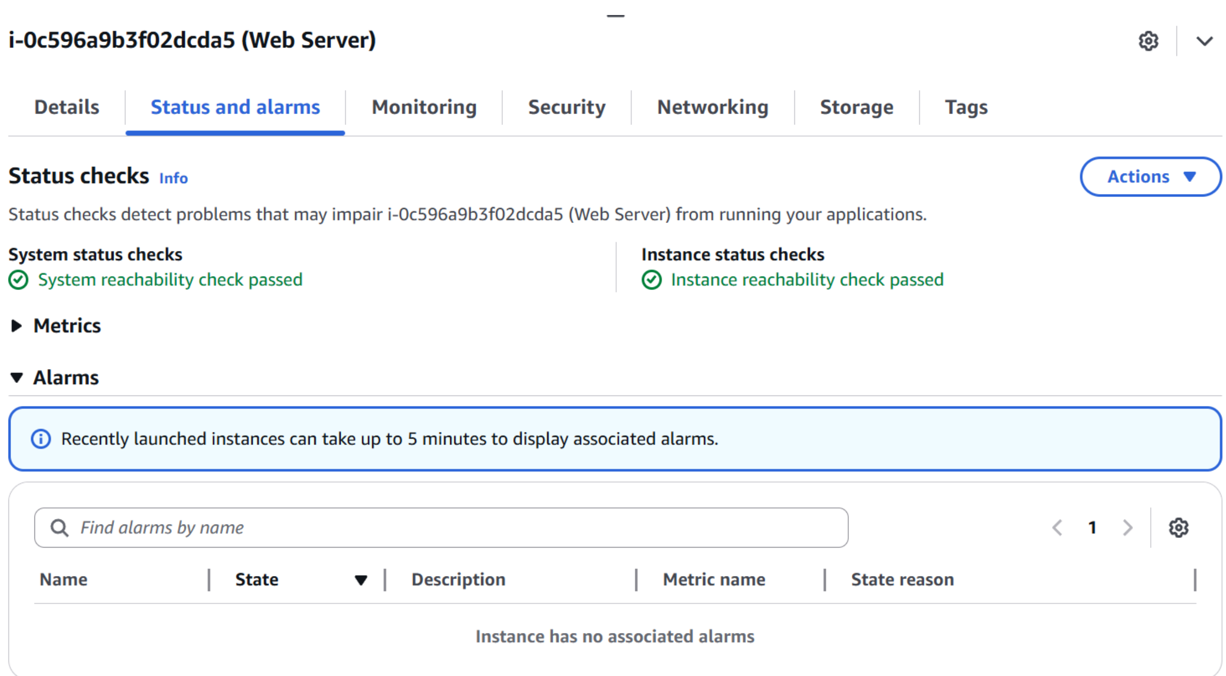



Now launch the instance by using the Launch instance button at the bottom of the console.

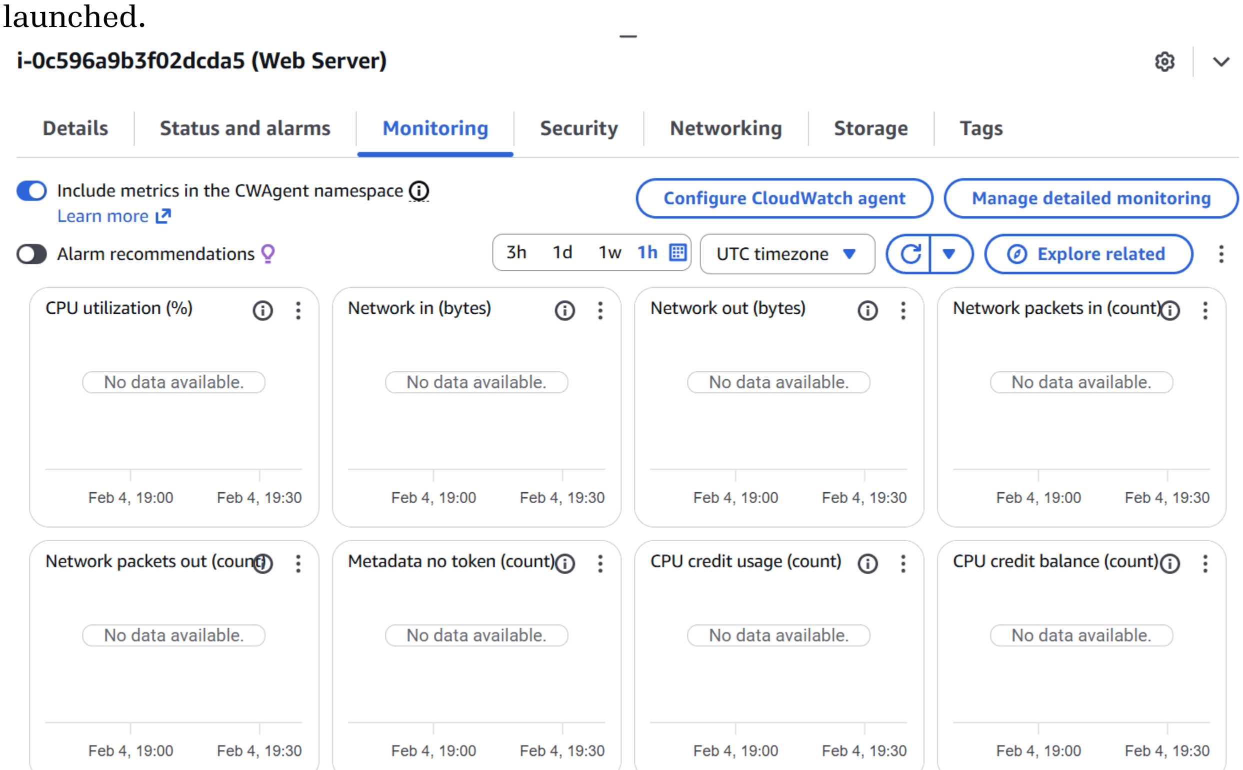
Now once the instance is created choose the View all Instances Button and select the Web Server you just created, it will display all the information about the instance you just created, the public IPv4 DNS is what you can use to connect to the Web Server.

Now Wait for the instance to pass all status checks and for the instance state to say "Running"

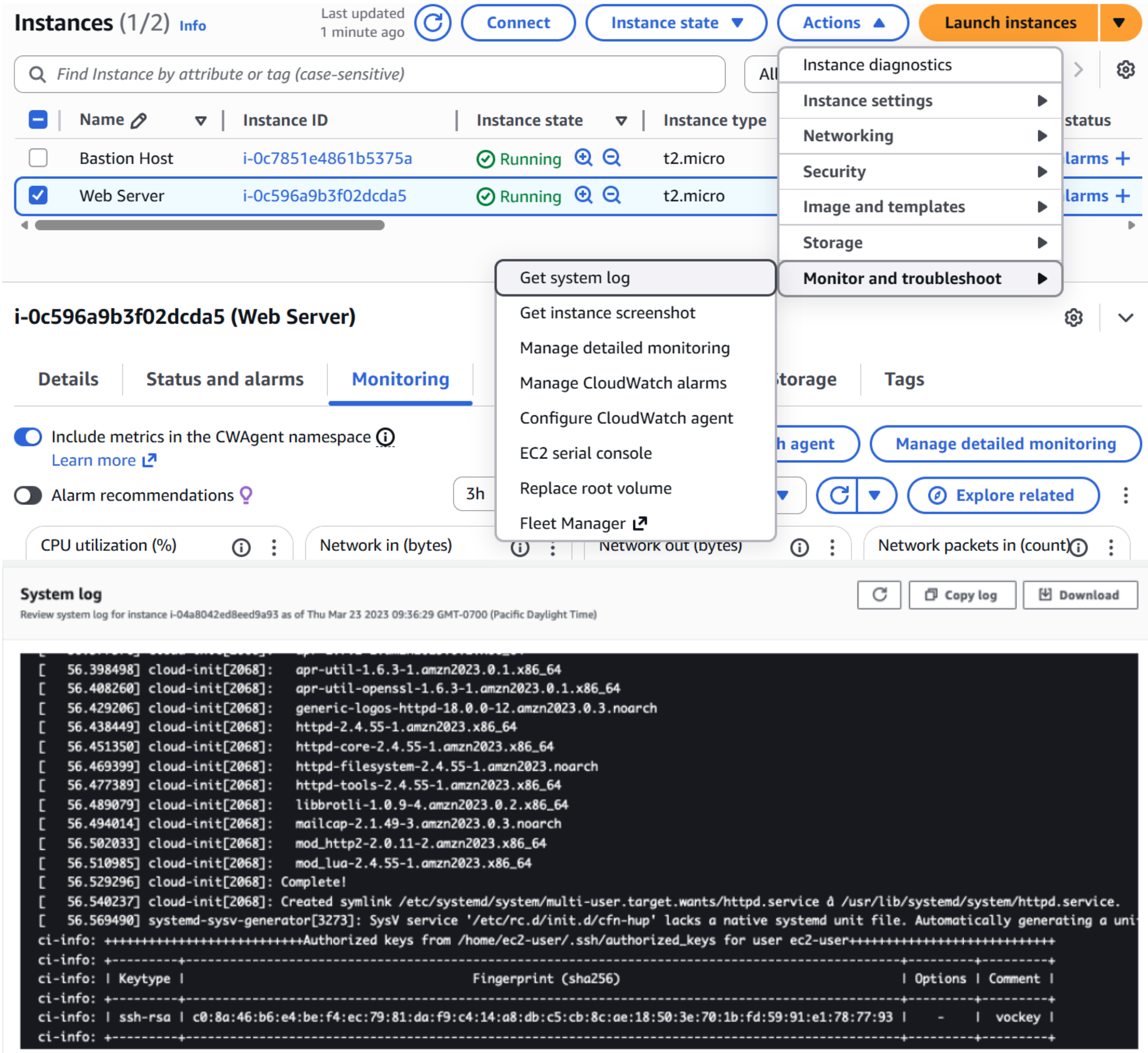
While the Instance is still selected navigate to the Status Checks tab and notice how the System reachability check and Instance reachability check passed.



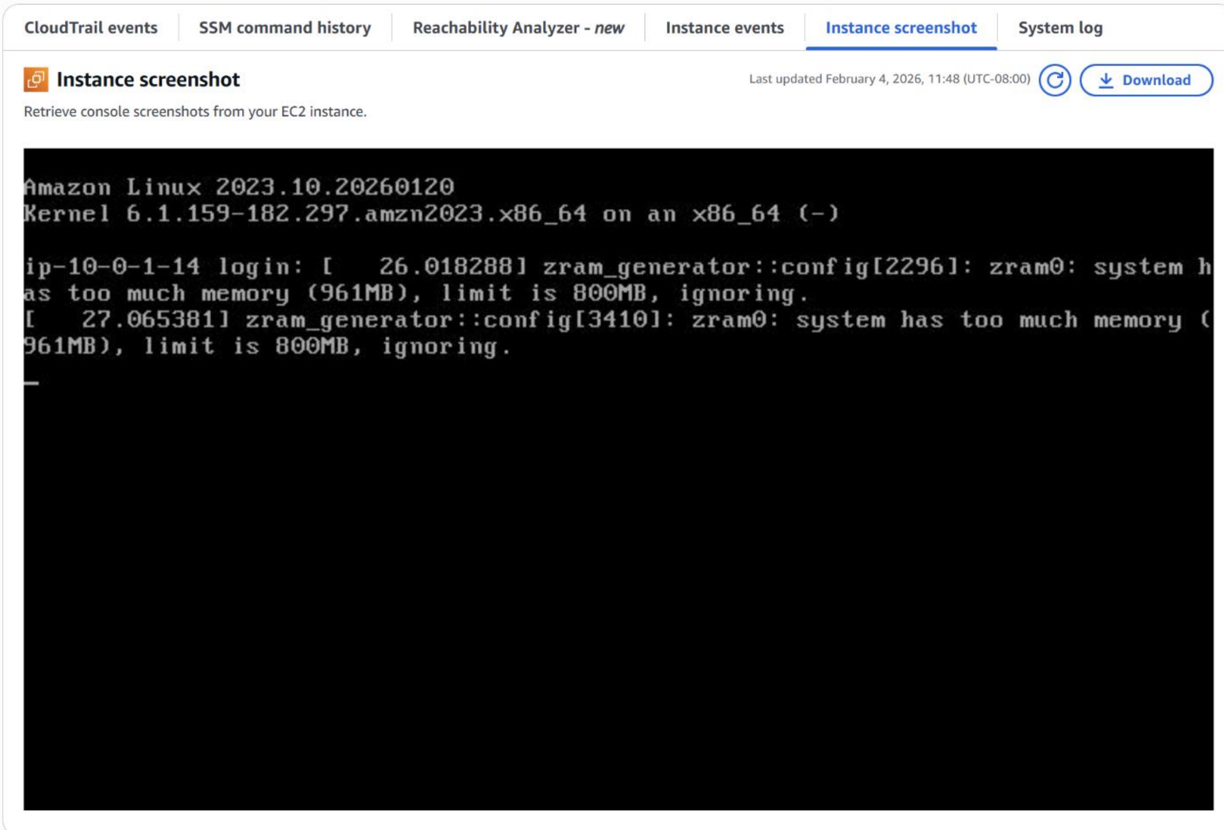

Now Choose the Monitoring tab, this displays all the CloudWatch Metrics for instance, you can choose the 3-dot icon on any graph to enlarge the data shown in the given metric. You will receive a No data available message if your instance was just launched.




In the Actions menu towards the top left of the console menu choose Monitor and Troubleshoot > Get System Log. This displays the system log and is important when troubleshooting Instance errors and malfunctions. When selecting this you can also see the HTTP package was installed in the instance from the code imputed into the user data.




Choose Cancel and make sure the Web Serve instance is still selected. Then in the Actions menu go to Monitor and troubleshoot > Get instance screenshot. This shows you what your instance looks like currently.




Make sure the Web Server is still selected and choose the Details tab to get the Public IPv4 address copied to your clipboard.

If you try pasting this URL you will notice it does not work, this is because of the security group we configured to deny access earlier. (Note that we did not configure any inbound rules for the security group which is why traffic is being denied as unless explicitly permitted, traffic is implicitly denied).

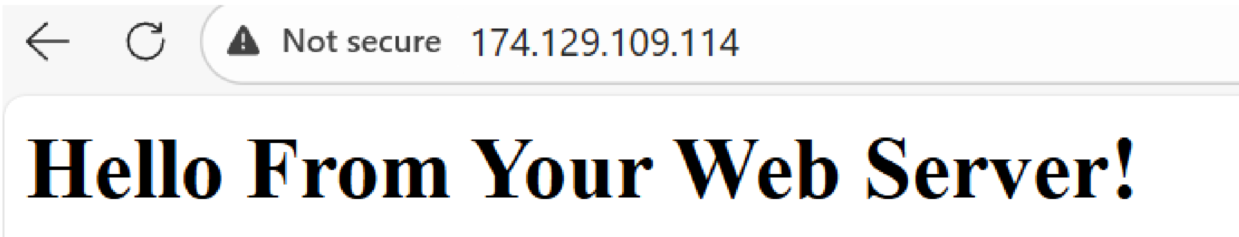
Keep the tab with the EC2 instance URL open and go back to the tab with the console. In the left navigation pane choose Security Groups.

Select the security group created during the instance Creation: Web security group.

Go to the inbound rules tab and select Edit Inbound rules, and select add rule and configure the rule for the following:

- Type: HTTP
- Source: Anywhere IPv4
- Choose: Save Rules

Go back to the tab open to the web server and refresh, you should see a message: *Hello From Your Web Server!*

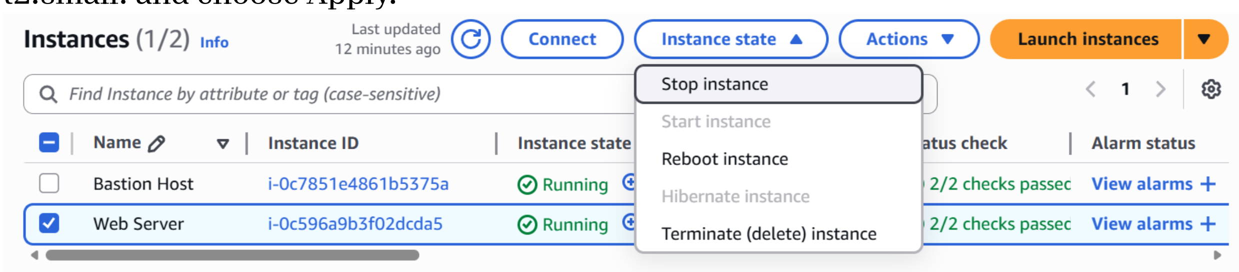


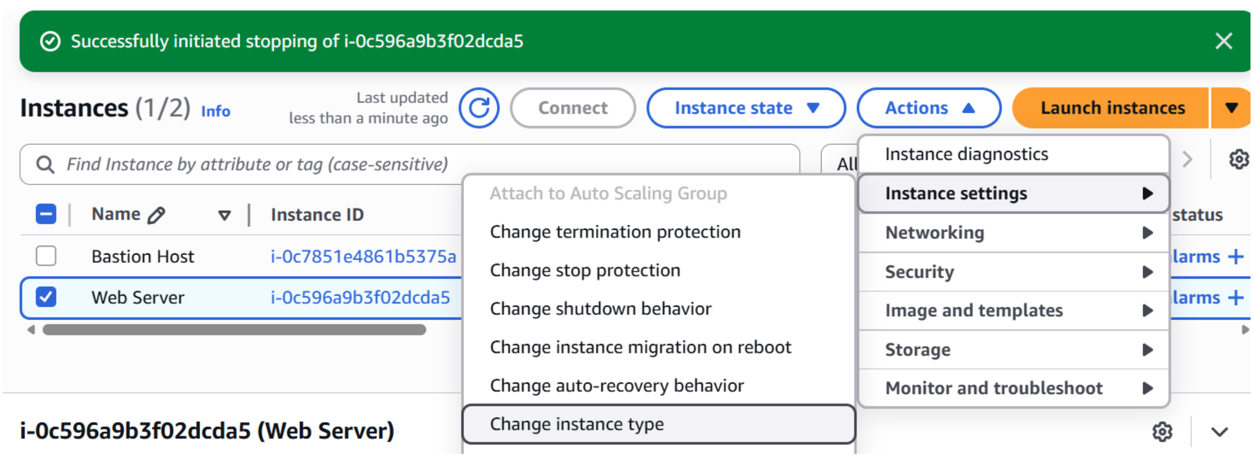

As your needs change, you might find that your instance is over-utilized (too small) or under-utilized (too large). If so, you can change the instance type.

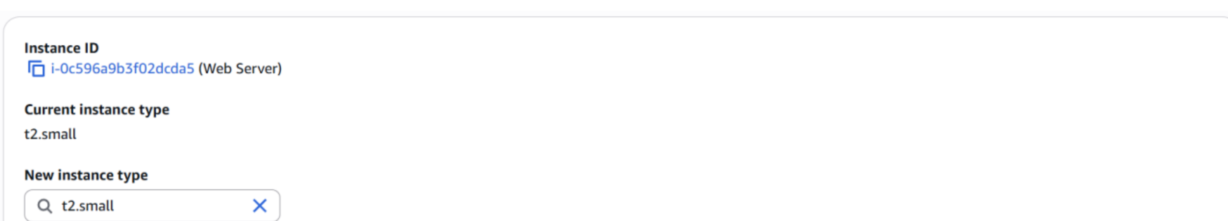
Before you can resize an instance, you must stop it.

In the EC2 Console go to the Instance State Menu and select Stop Instance while the Web Server Instance is selected. Wait for the Instance to display stopped.

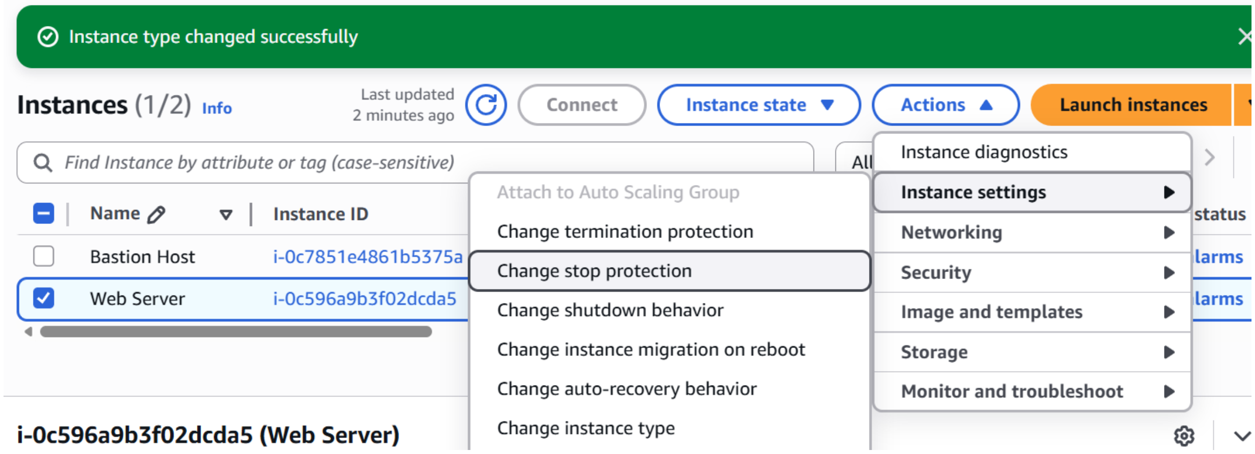
Keep the Web server Instance selected and go to the actions Menu and select Instance Settings > Change Instance Type, and configure the instance type to be t2.small. and choose Apply.



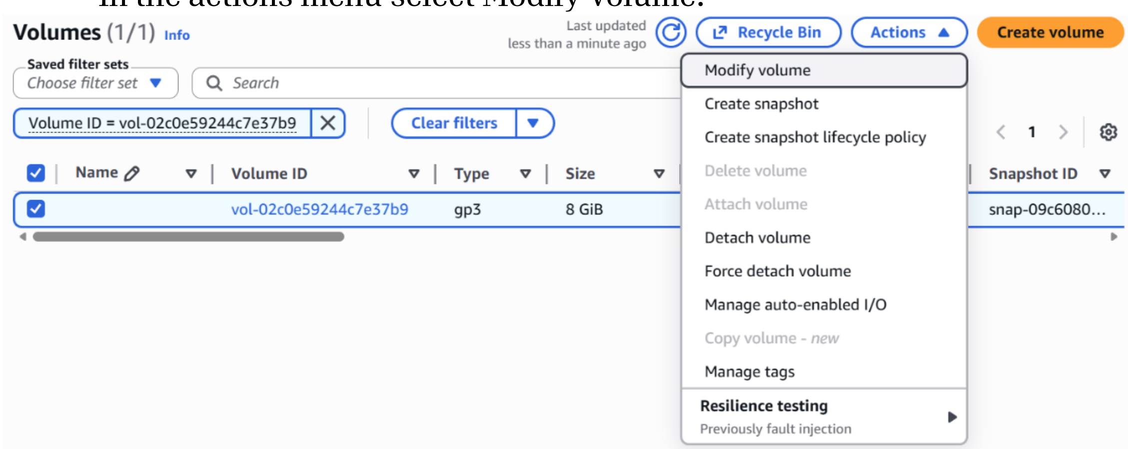

Select the Web Server instance, then in the Actions menu, select Instance settings > Change stop protection. Select Enable and then Save the change.



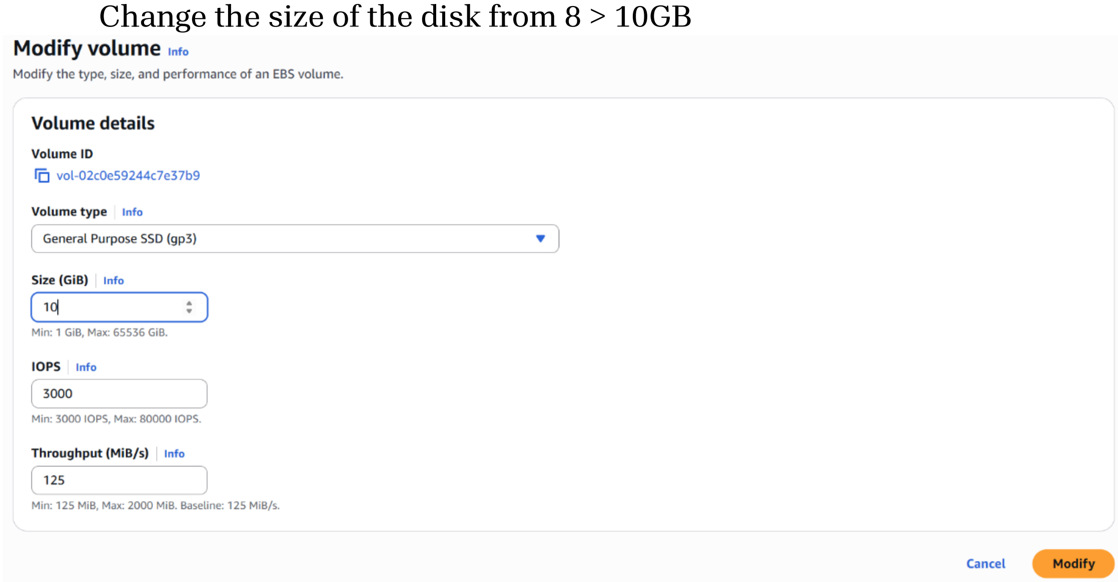

Now if you need more storage in the EC2 instance you can resize the EBS or Elastic Block Store Volume.

With the Web Server Instance still selected go the storage tab and select the Volume ID listed.

In the actions menu select Modify Volume.

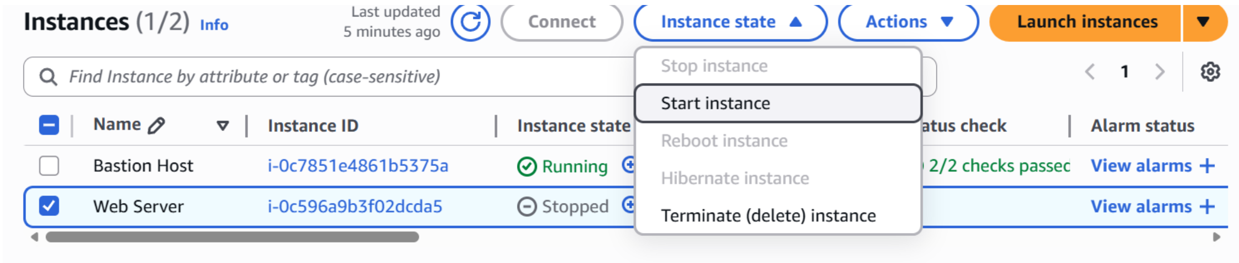



Change the size of the disk from 8 > 10GB




Choose Modify, and confirm your choice.

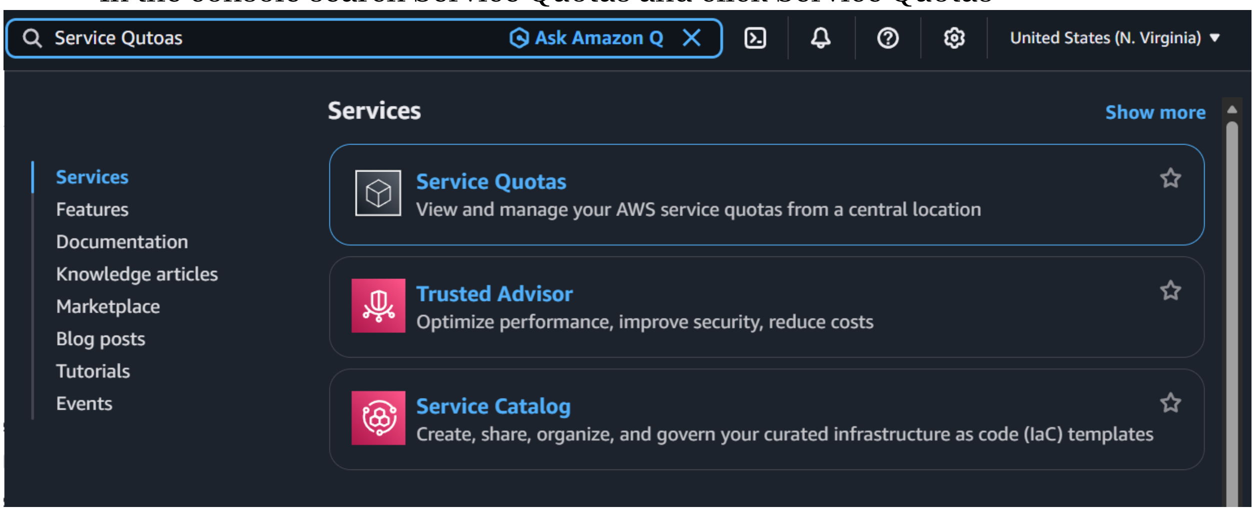
Now Select the Web server instance and start it



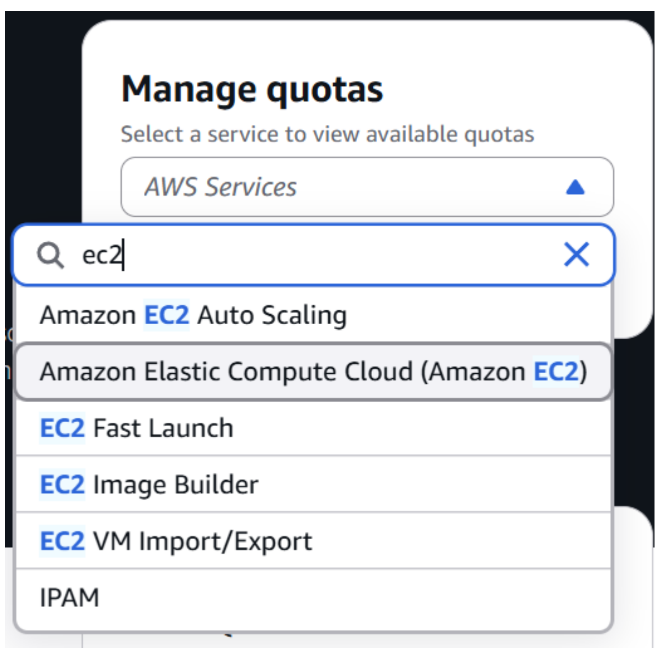

You have now resized the instance type and volume of the EC2 instance

When you create an AWS account there are per region limitations on how many EC2 instances you can own. In this next part we will explore those limitations.

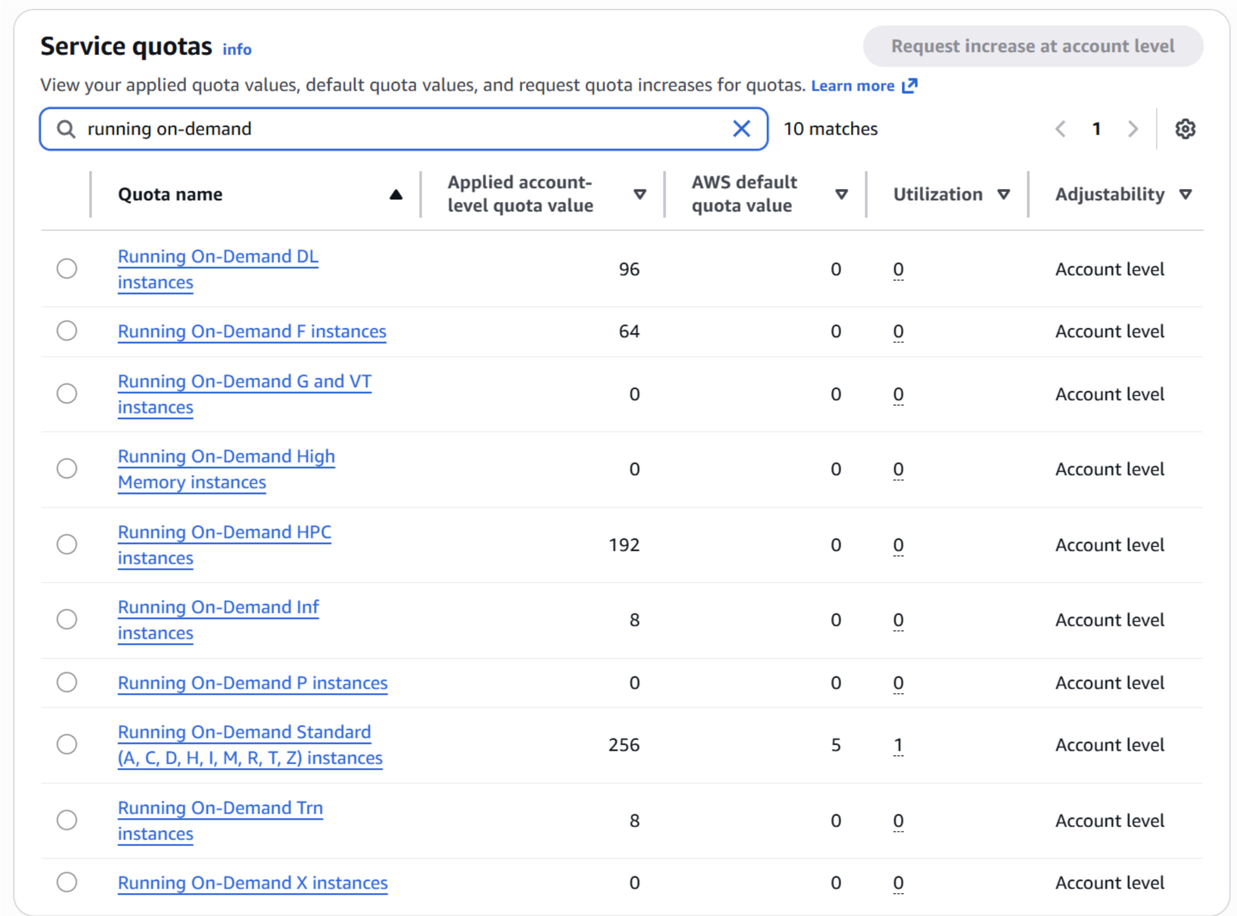
In the console search Service Quotas and click Service Quotas




In the Manage Quotas box search EC2 and choose Amazon Elastic Compute Cloud.

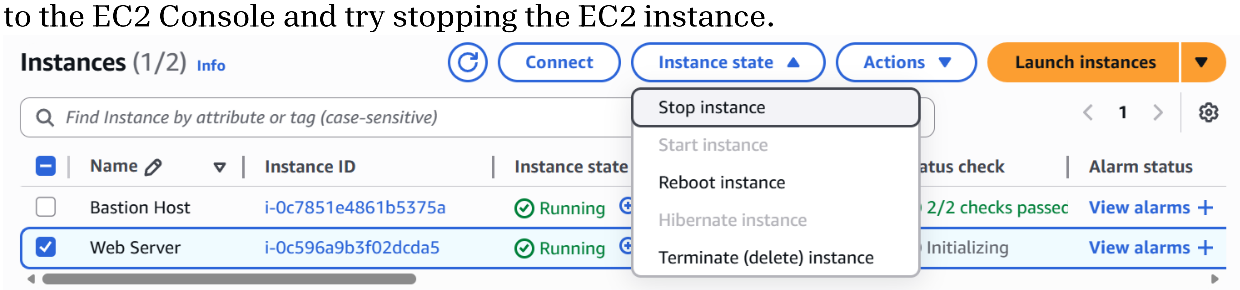


Now search running on-demand in the service quotas search box and just notice all the account limitations that the AWS account has for on-demand EC2 Instances like the Web Server EC2 instance you created.

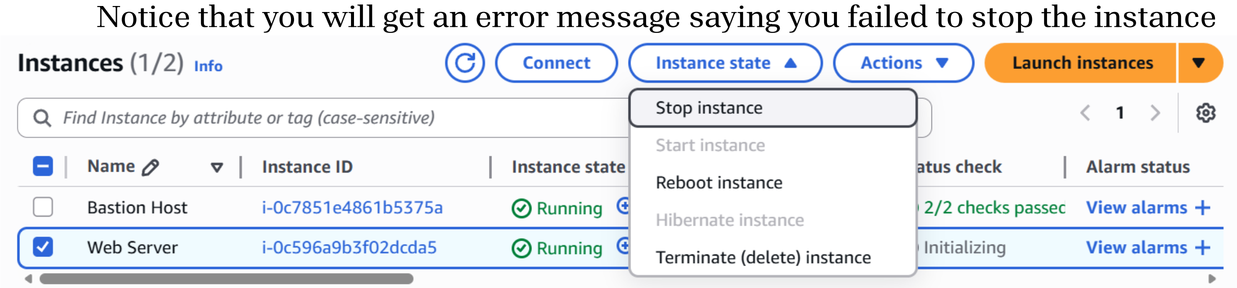



| Quota name | Applied account-level quota value | AWS default quota value | Utilization | Adjustability |
|---|---|---|---|---|
| Running On-Demand DL instances | 96 | 0 | 0 | Account level |
| Running On-Demand F instances | 64 | 0 | 0 | Account level |
| Running On-Demand G and VT instances | 0 | 0 | 0 | Account level |
| Running On-Demand High Memory instances | 0 | 0 | 0 | Account level |
| Running On-Demand HPC instances | 192 | 0 | 0 | Account level |
| Running On-Demand Inf instances | 8 | 0 | 0 | Account level |
| Running On-Demand P instances | 0 | 0 | 0 | Account level |
| Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances | 256 | 5 | 1 | Account level |
| Running On-Demand Trn instances | 8 | 0 | 0 | Account level |
| Running On-Demand X instances | 0 | 0 | 0 | Account level |

Now let's test the stop protection on the EC2 instance we created. Navigate back to the EC2 Console and try stopping the EC2 instance.

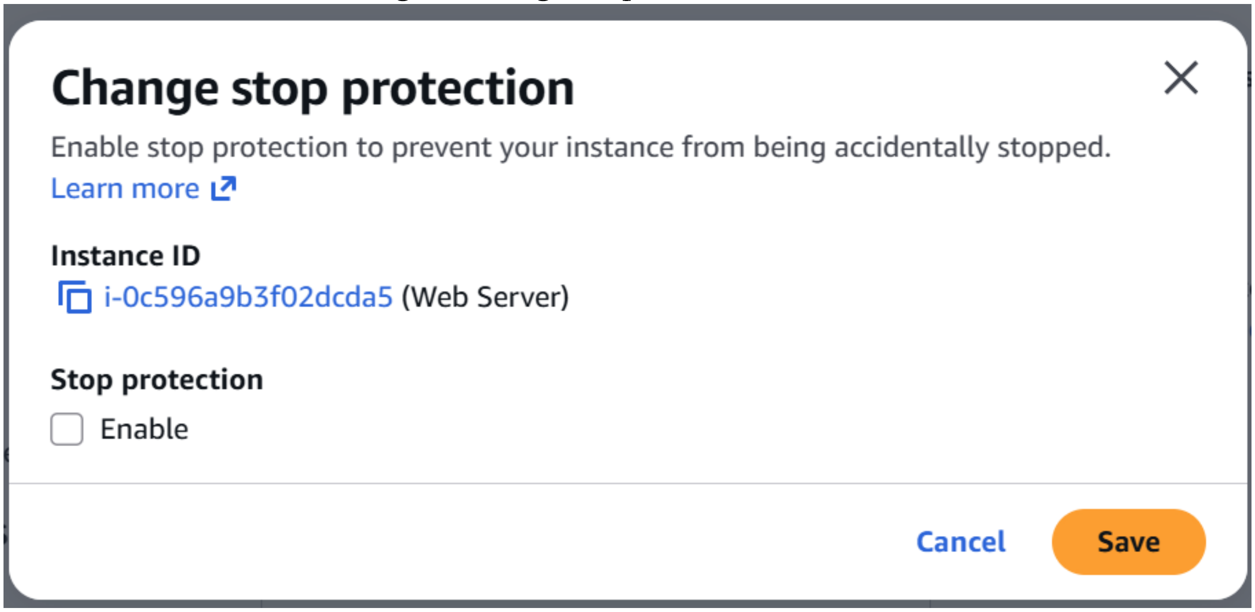



Notice that you will get an error message saying you failed to stop the instance



No remove the Stop protection the same way you enabled it but this time unchecking the enable checkbox.

Go to Instance settings > Change Stop Protection.




Now you've completed your lab, submit to the lab to receive a full score.

| Total score | 25/25 |
|---|---|
| Task 1 - EC2 instance created correctly | 5/5 |
| Task 2 - get system log requested | 5/5 |
| Task 3 - security group updated | 5/5 |
| Task 4 - EC2 instance updated | 5/5 |
| Task 6 - Instance stopped on second try | 5/5 |

**Conclusion**

This lab has helped develop the foundation of how to develop cloud solutions using AWS services like IAM, EC2, and VPCs. This additionally helps to build fundamental concepts within cloud computing like on-demand computing access and cloud security using the principle of least privilege. This lab additionally helps to get an understanding of how to configure and create cloud solutions using services other than AWS like Microsoft Azure. All in all, these 3 labs served as a great introduction to how to configure and create cloud solutions.